Linked Analysis Workspaces in Datalab

A PBUI- and CLIM-inspired design for coordinating dataset, pipeline, table, encoding, and chart views

August 2, 2026

Contents

Status: Architecture and interaction study
Analyzed codebase: the enhanced `pbui` repository and its `packages/datalab-ui` application supplied with this conversation
Date: 2026-08-02
Scope: data-model design, presentation semantics, interaction design, persistence, workspace duplication, source retargeting, performance, migration, and testing
Implementation status: the current document-linking implementation is analyzed as existing code. All snippets labeled **Proposed** are designs, not code already present in the repository.

Executive summary

The requested behavior should not be implemented as a set of peer-to-peer links among a table, chart, pipeline editor, encoding editor, and source browser.

Those views are not five independent copies of state. In the current Datalab model, the central object is already a `GraphicDocument`. It contains the source, transforms, output relation, mark, encodings, scales, and analysis settings. The chart, table, pipeline, and encoding applications are four different presentations of that one analytical composition.

The correct abstraction is therefore:

> **Several application views observe one shared analysis-selection subject.**

The enhanced repository already implements the first useful version of this idea. Distinct logical views may share a `documentBindingId`, and changing one view's selected document changes every member of that binding group. This already supports the simplest form of the requested workflow:

```
chart view ─────┐
table view ─────┤
pipeline view ──┼── shared document-selection subject ── GraphicDocument α
encoding view ──┘
```

Switch the shared subject from document α to document β, and all four views update together. Because each document owns its own source, transform pipeline, and encoding, selecting document β also selects its pipeline and encoding.

That implementation is a sound base, but it is not yet a complete linked-workspace system. The main gaps are:

1. The shared subject is denormalized into every `AppView`, so updates scan and rewrite all views.
2. The current link target is an entire tile, not a named application port.
3. The source browser is a singleton catalog with local React state, not a document-bound dataset view.
4. Replacing a source currently deletes the transforms, encoding, and parameters. It cannot preserve a work setup across compatible datasets.
5. Pipeline-step presentations carry only a step ID and dispatch against the ambient active document. They are not owner-qualified.
6. "Duplicate workspace" clones only layout geometry. It preserves the same logical views, document bindings, and documents, so the copy is a mirror rather than an independent workspace.
7. Stored templates are exact snapshots. They do not expose named inputs that can be bound to another analysis or dataset.
8. Portable bundles preserve current link equivalence, but the remote workbench protocol does not.

The recommended architecture has five parts.

1. Keep `GraphicDocument` as the analysis composition

Do not immediately split dataset, pipeline, relation, encoding, and plot into independent synchronized artifacts. Their dependency is directional:

```
source
  ↓
transforms
  ↓
result relation
  ↓
mark + encodings + scales
  ↓
plot
```

The UI relationship is different. It is a symmetric selection relationship:

```
dataset view ─────┐
pipeline view ────┤
table view ───────┼── analysis binding ── GraphicDocument
encoding view ────┤
chart view ───────┘
```

Keeping these two graphs separate is the central design decision.

2. Normalize typed subjects and application ports

Introduce first-class `SubjectBinding` records and let each logical application view connect named ports to them.

```ts
// Proposed
interface SubjectBinding {
  id: BindingId;
  kind: "analysis";
  value: { documentId: DocId | null };
  label?: string;
  revision: number;
}

interface AppView {
  id: ViewId;
  appId: AppId;
  bindings: Record<string, BindingId>;
  title?: string;
}
```

The chart, table, pipeline, encoding, and new dataset application each expose a `primary` analysis port. A future comparison application can expose `left` and `right` ports. A link group is not a special container; it is the derived set of ports that reference the same binding.

3. Use PBUI presentations for linking and rebinding

Add presentations for analysis bindings, analysis ports, owner-qualified pipelines, encodings, relations, sources, and steps. Use PBUI input contexts to select compatible visible ports or visible objects that can be converted to their owning analysis.

The chain icon becomes a normal way to start a presentation-based command:

```
Link this analysis port…
    → click another compatible analysis-port presentation

Use this pipeline's analysis in “Regional population”…
    → pipeline presentation converts to its owning analysis document
    → the selected binding is rebound atomically
```

Prepared selectors make eligibility checks fast, semantic identity prevents duplicate candidate work, and action rules contribute common link actions across presentation types.

4. Distinguish switching, retargeting, and forking

These are different operations and must not share one ambiguous “load” command.

- **Switch linked group to another analysis:** point the binding at another existing `GraphicDocument`. All linked views adopt that document’s source, pipeline, table output, and encoding.
- **Replace source and reset setup:** the current destructive behavior. It should remain available but be named honestly.
- **Fork linked analysis onto another source:** clone the document, preserve transforms and encodings, preflight it against the target schema, request field mappings where needed, compile it, and only then rebind the linked group to the new document.

For reusable workspaces, **fork** should be the default source-changing operation because it preserves the original analysis.

5. Give workspace duplication explicit graph semantics

The current geometry-only operation should be called **Mirror workspace**. A user-facing **Duplicate workspace** should normally clone the entire reachable graph:

- layout nodes;
- logical views;
- binding equivalence classes;
- documents;

- internal sharing relationships.

The portable bundle exporter/importer already contains most of the required graph-copying logic. It should be reused rather than reimplemented.

Templates should then evolve from saved snapshots into parameterized workspace definitions with named analysis slots. A “Regional population analysis” template can expose one slot called `Main analysis`; chart, table, pipeline, encoding, and dataset ports all bind to that slot. Instantiation can bind the slot to an existing document or fork a blueprint document onto a selected source.

1. The concrete problem

Consider an analytical workspace with five tiles:

1. a dataset summary;
2. a transform pipeline;
3. a table showing the pipeline result;
4. an encoding editor;
5. a chart.

The repository's census fixture contains `region`, `population`, `area_km2`, and `station_id`. A repository-accurate version of the user's example is therefore:

```
Dataset: lab / census / version 2 / rows.csv

Pipeline:
    group by region
    sum population as population_total

Table:
    region | population_total

Encoding:
    mark = bar
    x = region
    y = population_total
    color = region

Chart:
    Population by region
```

The user wants the workspace to behave as a reusable analytical instrument rather than as five unrelated selectors.

A successful system supports at least these workflows.

1.1 Coordinated switch to an existing analysis

The user selects a different saved analysis from the pipeline tile. Every linked view changes to that analysis:

```
Before:
    binding A → “Population by region”

After:
    binding A → “Temperature by station”
```

The dataset tile shows the climate source, the pipeline editor shows the climate transforms, the table shows the climate result, the encoding editor shows line-chart encodings, and the chart redraws.

1.2 Coordinated switch initiated from any member

The chart is not the “master.” Neither is the pipeline. A change initiated from any port changes the one shared subject. The direction of interaction is symmetric even though the underlying analytical dataflow is directional.

1.3 Apply the same setup to another compatible dataset

The user duplicates the workspace and selects a newer census source. They want the copied pipeline and encoding to survive when the schema is compatible.

The original workspace must remain pointed at census version 2. The duplicate may point at census version 3.

1.4 Adapt the same setup to a structurally similar dataset

The target source may use different names:

```
old source                  new source
----------                  ----------
region                      city
population                  residents
area_km2                    land_area
```

The system should discover that a field mapping is required, ask the user once, rewrite source-origin references, compile the candidate document, and switch the group only after successful validation.

1.5 Save the workspace as a reusable tool

The user saves “Population aggregation” as a template. The saved definition should preserve:

- the layout;
- the kinds of applications;
- which application ports are linked;
- the pipeline and encoding blueprint;
- the semantic requirements of the input;
- optionally, a default source.

Instantiating the template should ask for one analytical input, not require five separate dropdown changes.

These workflows reveal that “linking views” is partly a selection problem, partly a graph-copy problem, partly a schema-adaptation problem, and partly a presentation-command problem. Treating it as only a chain icon and one shared string ID will solve the first demo but leave the reusable-workspace goal

incomplete.

2. Terminology and mental model

The following terms are used consistently throughout this study.

2.1 Application object

An object in the domain model that the user can refer to meaningfully: a document, source, field, transform step, output relation, encoding, analysis binding, workspace, or tile placement.

An application object need not be a JavaScript object with unique allocation identity. Two separately constructed references may denote the same field or the same binding. PBUI semantic identity answers that question.

2.2 Presentation

A visible occurrence of an application object, annotated with a presentation type. A field chip in a table header, a pipeline step label, an analysis chip in a toolbar, and a source row can all be presentations.

The same object may have many presentations. One presentation occurrence may be clicked while another occurrence of the same semantic object is elsewhere on screen.

2.3 Logical application view

An open chart, table, pipeline editor, encoding editor, or other application configuration. In the current code this is `AppView`, keyed by `ViewId`.

A logical view is not the same as a workspace rectangle. Multiple placements can present one logical view.

2.4 Placement

One leaf in a workspace layout tree: a rectangle that presents a logical application view. In current code the leaf has its own node ID and a `viewId`.

2.5 Analysis composition

The complete analytical specification that turns a source into an output and a visual representation. In the current model this is a `GraphicDocument`.

It contains:

- source nodes;
- transforms;
- views;
- the root view;
- relation references;
- mark;

- encodings;
- scales;
- analysis settings;
- parameters.

The product UI may call this an **Analysis**, even if the internal type remains `GraphicDocument`.

2.6 Subject

A value that one or more application ports observe. Initially the only proposed subject kind is `analysis`, whose value is a selected `DocId`.

A subject is deliberately narrower than a view. Sharing the analysis subject does not automatically share zoom, cursor position, table sorting, row selection, or panel expansion state.

2.7 Binding

A first-class record that stores a subject value and has stable identity. Several ports are linked when they reference the same binding.

2.8 Port

A named connection point on a logical application view. A normal chart has one `primary` analysis port. A comparison view might have `left` and `right` analysis ports.

Ports are important because “link this tile” becomes ambiguous as soon as one tile can observe more than one subject.

2.9 Rebind

Change the value of an existing binding. Every port connected to it observes the new value.

2.10 Merge bindings

Make the members of two formerly separate groups refer to one binding. A deterministic source-wins rule resolves the subject value at merge time.

2.11 Detach

Disconnect one port from a shared binding by creating a fresh private binding initialized to the same current value. Detachment should not cause a visible jump.

2.12 Retarget

Change an analysis document’s source while attempting to retain its pipeline and visual specification.

2.13 Fork

Create independent identities while preserving structure. Forking an analysis produces a new document. Forking a workspace produces new layout, view, binding, and document identities while preserving internal sharing relationships.

---

3. Presentation-based UIs and the relevant CLIM pattern

A presentation-based UI does not treat visible text and shapes as inert pixels. It remembers the application objects represented by those pixels.

That distinction changes command design.

In a conventional event handler, a chain button might open a bespoke tile picker. The picker returns a tile ID. The command then performs hand-written checks.

In a presentation-based design, the button establishes an input context:

> Accept a visible presentation denoting an analysis port compatible with this port, excluding the current group.

Every visible eligible presentation becomes an input target. The application does not need to know whether the target appeared in a tile toolbar, a workspace inspector, a group manager, or a breadcrumb. It needs only the presentation type, semantic object, and current input context.

3.1 The relevant CLIM ideas

Common Lisp Interface Manager systems distinguish several concepts that are useful here:

- **application objects** are domain values;
- **presentation types** describe how an object may participate in interaction;
- **presentations** associate visible output with an object and type;
- **input contexts** specify what type of object a command currently wants;
- **presentation translators** interpret an object of one presentation type as an input or command argument of another;
- **commands and command tables** organize actions independently from the exact widget that displayed the object;
- **partial commands** allow a command to be initiated before all arguments have been supplied.

The goal is not to reproduce CLIM's Lisp protocol literally. The useful pattern is the separation of object, presentation, input requirement, conversion, and command.

PBUI now has direct counterparts for much of this:

CLIM-oriented concept	PBUI mechanism
presentation type	key in `PresentationValues`
presentation occurrence	`Present`/presentation reference
semantic object identity	descriptor identity domain and key

CLIM-oriented concept	PBUI mechanism
input context	`pbui.accept(...)`
input restriction	`PresentationSelector`
prepared type/predicate logic	selector `prepare`
translator	named weighted conversion
command contribution	descriptor actions and action rules
command execution	serializable verb application

The linked-workspace design should use these mechanisms instead of creating a second, unrelated selection framework.

3.2 Why object identity matters for links

A binding chip may be rendered in five toolbars. These are five occurrences of one application object.

PBUI should know that they are semantically identical:

```
// Proposed identity
{
  domain: "analysis-binding",
  key: binding.id,
}
```

Identity-based selector memoization can then evaluate group-level eligibility once per binding rather than once per occurrence. More importantly, the system can highlight all occurrences as representations of one accepted object while still returning the exact clicked occurrence's payload when the user commits.

3.3 Why conversions matter

A user may click a pipeline presentation while a command asks for an analysis. The pipeline is not a subtype of an analysis document; it is a component owned by one. A conversion can express that interpretation:

```
pipeline(doc α) ── owns-analysis ──> doc α
```

The same is true for an encoding, output relation, owner-qualified source node, or step. This lets "use the analysis represented by what I clicked" work across the interface without teaching every caller about every possible visible object.

3.4 Why partial commands matter

A chain icon initiates a command whose target is not yet known:

```
link-analysis-port(sourcePort, ?targetPort)
```

The existing `pbui.accept` call is sufficient for the first implementation. A more complete PBUI command host can later represent this as a serializable partial command, display the missing argument in the command UI, accept a presentation, and then dispatch the completed verb.

That is a productive direction because it makes linking available from menus, keyboard commands, command palettes, and tutorials without embedding asynchronous selection logic in each component.

---

4. Two graphs, not one

The most important modeling distinction is between the **analysis dependency graph** and the **UI binding graph**.

4.1 Analysis dependency graph

An analysis computes and presents information in one direction:

```
SourceRef
   │
   ▼
source relation
   │
   ▼
transform 1
   │
   ▼
transform 2
   │
   ▼
output relation
   │
   ├──────────────► table
   │
   ▼
mark + encodings + scales
   │
   ▼
plot
```

A change to an upstream source can invalidate downstream field references. A change to an aggregate output can invalidate an encoding. This is a dependency graph with validation and compilation rules.

4.2 UI binding graph

Application views observe a selected analysis:

```
dataset.primary ─────┐
pipeline.primary ────┤
table.primary ───────┼── binding A ── document α
encoding.primary ────┤
chart.primary ───────┘
```

There is no pipeline-to-chart event and no table-to-encoding event. Every member reads the same subject.

When the binding changes:

```
binding A: document α → document β
```

all views render document β.

4.3 Why peer-to-peer synchronization fails

Suppose a chart, pipeline, table, and encoding editor each hold their own `docId`, and changes are propagated by events:

```
pipeline changed → chart
pipeline changed → table
pipeline changed → encoding
chart changed → pipeline
chart changed → table
...
```

This quickly creates:

- ordering problems;
- event loops;
- partially updated UI;
- stale updates;
- unclear conflict rules;
- quadratic wiring;
- difficulty adding a fifth view;
- difficulty undoing one logical operation;
- ambiguity when one tile has two subjects.

A binding reduces the operation to one update of one subject record.

4.4 Why one global active document also fails

A global active document removes wiring but destroys locality.

It prevents:

- two independent analytical groups in one workspace;
- side-by-side comparison;
- a chart following one analysis while another table follows another;
- independent duplicated workspaces;
- stable background tiles when focus changes;
- commands that refer to the object actually clicked rather than ambient focus.

The current Datalab code still has some ambient `activeDocId` fallbacks. They are useful for ownerless global commands, but owner-qualified presentations should not rely on them.

4.5 Why one omnibus “link group” also fails

It may be tempting to put every shared state facet behind one `linkedGroupId`:

```
document
time range
row selection
cursor
zoom
filters
color scale
```

That creates accidental coupling. Users may want two charts to share the analysis and filter set but keep independent zoom. A comparison view may share a time range across both sides while observing two analyses.

Typed subjects keep these choices independent:

```
analysis binding A
filter binding F
time-range binding T
cursor binding C
```

This study implements only the first kind. The architecture should make adding the next kind possible without changing the meaning of the first.

---

5. Current Datalab architecture

This section records what the supplied enhanced repository already does.

5.1 Logical views, placements, and document bindings

`packages/datalab-ui/src/store/layout.ts` defines:

```
export interface AppView {
  id: ViewId;
  appId: AppId;
  documents: Record<string, DocId>;
  documentBindingId?: string;
  title?: string;
}
```

This is already a substantial improvement over storing application state directly in a layout leaf.

The identities are:

```
layout node ID      placement geometry
view ID             logical application view
document ID         analytical composition
binding ID          shared document-selection subject
```

A missing `documentBindingId` falls back to the view ID for backward compatibility.

`setViewDocument` finds every view with the same effective binding ID and writes the selected document role into each member. `linkViewDocuments` merges two groups, creates a fresh binding ID, and copies the source group's document-role map into every merged member. `unlinkViewDocuments` gives one view a fresh private binding while preserving its current document map.

The source-wins rule is deterministic and sensible for a direct command initiated from a source view.

5.2 Exactly four current document-bound applications

The application registry documents that chart, table, pipeline, and encoding are document-bound because they are "views of one composition."

Each application reads `view.documents.primary`:

```
ChartApp      → useDocPlot(docId)
TableApp      → useDocAnalysisResult(docId)
PipelineApp   → useDocAnalysisResult(docId)
EncodingApp   → useDocAnalysisResult(docId)
```

Each renders `DocBar`, so each can change or link its selected document.

This means the current link feature already coordinates the four core analytical views.

5.3 `GraphicDocument` already owns the requested parts

`packages/datalab-ui/src/model/graphic.ts` defines `GraphicDocument` with:

- `sources`;
- `transforms`;
- `views`;
- `rootView`;
- `parameters`;
- metadata and identity.

An authoring view contains its relation, mark, encodings, scales, analysis settings, and other visual configuration.

For the current product model, “the pipeline,” “the encoding,” and “the plot” are not independent top-level store records that need synchronization. They are projections of a document.

That makes a document-level analysis binding the correct first subject.

5.4 Current chain interaction

`DocBar` starts a PBUI input context:

```
const result = await pbui.accept({
  prompt: "LINK DOCUMENT SELECTION — click the title of another document-bound tile",
  selector: presentationSelectors.type("tile", {
    cache: "identity",
    prepare: () => (tile) =>
      tile.docBound &&
      tile.viewId !== viewId &&
      tile.documentBindingId !== bindingId,
  }),
});
```

The selector is prepared once for the accept operation. It accepts a different document-bound tile outside the current group. The chosen tile’s `viewId` is then passed to `linkViewDocuments`.

This is a good demonstration of the enhanced PBUI selector protocol.

5.5 Current source browser

`SourceApp` is:

- `docBound: false`;

- `singleton: true`;
- backed by local React state for chosen drop and dataset;
- a catalog browser over server resources.

It is not a view of the selected document's root source. Consequently it cannot simply join the current document-binding group.

That is not necessarily a defect in `SourceApp`; a catalog and a selected-source inspector are different applications. The missing piece is a document-bound dataset application.

5.6 Current source replacement is destructive

`worldActions.setDocSource` explicitly states that a new source invalidates pipeline and encoding. It calls `replaceDocumentSource`, which resets:

```
document.transforms = {};
document.views = replacement.views;
document.rootView = replacement.rootView;
document.parameters = {};
```

This is honest for an arbitrary incompatible source, but it cannot support "reuse this work setup with another dataset."

A separate preflighted fork/retarget operation is required. The current command should not silently change semantics.

5.7 Current pipeline-step ownership gap

The presentation vocabulary currently defines:

```
step: string;
```

`PipelinePanel` presents only `step.id`, while `stepDescriptor` sends verbs to `env.activeDocId`.

This creates a correctness hazard. A step visibly rendered in pipeline document β may dispatch against ambient active document α.

The type should become owner-qualified:

```
// Proposed
interface StepRef {
  docId: DocId;
  stepId: string;
}
```

This is not only a bug fix. It makes the step convertible to its owning analysis and therefore useful in linked-workspace commands.

5.8 Current workspace clone is a mirror

`cloneTree` mints new layout-node IDs but preserves each leaf's `viewId`. `cloneSpace` copies only the tree and workspace metadata.

Therefore the current "Duplicate workspace" behavior is:

```
workspace α tree ─┐
                  ├── same AppViews ── same bindings ── same documents
workspace β tree ─┘
```

Changing the selected document, pipeline, or encoding in the copy changes the original because both workspaces present the same logical views.

This behavior is useful, but it is a mirror, not the independent duplicate expected by the stated goal.

5.9 Portable bundles already perform graph-aware copying

`store/bundles.ts` exports documents and views through collectors that replace runtime IDs with dense indices.

The view collector also replaces runtime binding IDs with numeric equivalence-class indices. During import, new document, view, binding, and layout IDs are minted while internal sharing is reconstructed.

This already provides the key algorithmic property needed for a full workspace fork:

> preserve aliases inside the copied graph while creating no aliases back to the source graph.

The implementation should be reused directly or refactored into shared graph-copy primitives.

5.10 Current templates are snapshots

`store/templates.ts` stores a portable `Bundle` verbatim.

This correctly preserves a workspace, but the template includes exact documents and source references. It has no named input slots, schema contracts, or binding prompts.

It is therefore a saved snapshot/preset rather than a parameterized analytical tool.

5.11 Remote protocol does not preserve link groups

The current protobuf `AppView` has:

```
message AppView {
  string id = 1;
  string app_id = 2;
  map<string, string> documents = 3;
  optional string title = 4;
}
```

There is no document-binding field. The remote codec explicitly treats link equivalence as a local/portable capability that is lost in a remote workbench round trip.

Any production linked-workspace feature needs a protocol extension.

5.12 Existing execution coordination is useful but incomplete

`AnalysisCoordinator` already provides:

- semantic request keys;
- in-flight coalescing;
- a bounded cache;
- generation tracking;
- stale-result rejection.

This reduces duplicate execution when several views request the same document result.

Compilation and default-view initialization still occur through hooks mounted in multiple applications. A linked five-view workspace should share a centralized per-document analysis resource so that one subject switch does not cause redundant compilation or competing initialization effects.

---

6. What the existing document binding already solves

It is important not to redesign what already works.

6.1 Four distinct applications can follow one document

A linked chart, table, pipeline, and encoding view preserve their distinct `viewId`, `appId`, title, and placement while observing one selected document.

This is exactly the core semantic required for coordinated analysis switching.

6.2 Switching to another existing document is already powerful

Because `GraphicDocument` owns its source, transforms, and encoding, selecting another document is not merely switching a dataset identifier. It selects a complete analytical composition.

For example:

```
document α
    source: census
    transform: aggregate population by region
    mark: bar
    x: region
    y: population_total

document β
    source: climate
    transform: filter station, rolling mean
    mark: line
    x: time
    y: temp_c
```

Rebinding the group from α to β causes all four applications to present the second setup.

6.3 Independent logical views remain possible

Ordinary view duplication produces a new logical view and a fresh binding. Linked placement duplication reuses the logical view. The model therefore distinguishes:

- another placement of the same view;
- another independent view of the same document;
- another view linked only at the selection-subject level.

That identity separation should be retained.

6.4 PBUI already supplies the selection machinery

The current chain interaction demonstrates:

- a presentation input context;
- a prepared selector;
- semantic identity caching;
- exact commitment of a selected occurrence;
- a reducer-level operation.

The next design should generalize the selected object from a whole tile to a named analysis port and generalize the accepted objects from only tiles to any presentation convertible to an analysis.

---

7. Current limitations and failure modes

The desired product experience exposes limitations beyond the initial document-binding patch.

7.1 Denormalized subject state

The current binding identity is shared, but its selected document map is copied into every member view.

Updating one role performs an `O(number of views)` scan and mutates every matching `AppView`.

More importantly, the state model permits disagreement:

```
view 1: binding A, primary doc α
view 2: binding A, primary doc β
```

Reducers currently maintain the invariant, but persistence bugs, external state construction, or future protocol code can create an invalid split-brain group.

A normalized binding record makes disagreement unrepresentable.

7.2 A whole tile is too coarse as a link endpoint

Today the chain interaction accepts `<tile>`.

That works while every document-bound application has exactly one document role. It becomes ambiguous for:

- a comparison app with `left` and `right`;
- a join editor with two inputs;
- a dashboard with several independently bound panels;
- an application that observes an analysis read-only but edits another;
- future filter, time-range, cursor, or row-selection subjects.

The selectable object should be an `analysisPort`, visually presented in the tile header.

7.3 `docBound: boolean` is not expressive enough

A boolean cannot describe:

- the port name;
- subject kind;
- whether the port is required;
- read-only versus read-write access;
- multiple ports;
- the label shown to users;

- whether a view can create a fresh subject;
- whether a port may remain unbound.

App descriptors need explicit port metadata.

7.4 “Document” is internally accurate but product-ambiguous

The toolbar currently says `Doc`.

To a newcomer, a document may mean a file, table, source, notebook, or chart. In this product the object is a complete analytical composition.

The UI should say **Analysis** or **Composition**. Internal `DocId` and `GraphicDocument` names can remain during migration.

7.5 Source changes and analysis switches are conflated

Selecting a different existing document is safe because that document carries a coherent pipeline and encoding.

Replacing the source inside the current document is potentially destructive because field references may no longer resolve.

These must be separate commands with separate previews and undo behavior.

7.6 No source-bound member view

The catalog browser is useful for discovering sources, but the linked workspace needs a tile that answers:

- Which source is the current analysis using?
- What schema does it expose?
- Is it compatible with a candidate source?
- Which fields are consumed by the current pipeline?
- Can I fork this linked analysis onto another source?

Without this view, the source is visible only indirectly through the analysis selector and other application details.

7.7 Ambient ownership weakens presentation commands

Bare `step` IDs and other ownerless values force descriptors to use `activeDocId`. That means actions do not necessarily target the object visibly presented.

Linked workspaces make this more dangerous because several analyses may be visible simultaneously.

Every presentation for an analysis-owned object should carry its owner identity.

7.8 Workspace copies preserve cross-workspace aliases

A user duplicating a workspace to apply the same setup to another dataset will unexpectedly alter the original. This is a direct mismatch with the stated goal.

The UI needs explicit copy semantics and names that match those semantics.

7.9 Templates cannot express “same tool, new input”

A saved bundle can reproduce the exact census analysis, but it cannot say:

```
This linked group needs one analysis input satisfying contract C.
Use this stored document as the blueprint if the user selects a source.
```

The absence of slots is the boundary between a snapshot and a reusable tool.

7.10 Deleting a referenced document can leave invalid references

World document deletion reassigns `activeDocId`, but linked view bindings can still contain a deleted document ID unless a higher-level invariant repair handles them.

Normalized bindings make affected groups easy to find. Deletion should either:

- refuse while bindings reference the document;
- ask for replacement;
- set affected bindings to `null`;
- or delete/fork the affected views as one explicit transaction.

Silent dangling references are not acceptable.

---

8. Requirements and invariants

A robust design begins with rules that must always hold.

8.1 Functional requirements

The system shall support:

1. linking compatible analysis ports;
2. detaching one port without changing its current visible analysis;
3. merging two existing groups with a deterministic winner;
4. rebinding a group from any member;
5. selecting an analysis through any visible presentation convertible to an analysis;
6. adding all compatible views in a workspace to one group;
7. showing group identity, label, member count, and current analysis;
8. switching to another complete existing analysis;
9. forking an analysis onto another source with validation;
10. mapping incompatible field names explicitly;
11. mirroring, independently duplicating, and fully forking workspaces;
12. saving and instantiating parameterized workspace templates;
13. preserving groups through local, portable, and remote persistence;
14. keyboard and screen-reader operation;
15. atomic undoable high-level operations.

8.2 State invariants

At every committed state:

1. Every application port references an existing binding.
2. A port’s declared subject kind equals its binding’s subject kind.
3. Every non-null analysis binding references an existing document.
4. One binding stores exactly one subject value.
5. Two linked ports are linked because they reference the same binding ID, not because their values happen to be equal.
6. Two private bindings may select the same document without being linked.
7. Detaching creates a new binding with the same value.
8. Merging has a documented source-wins or explicitly chosen winner.
9. Unreferenced bindings are garbage-collected.
10. A linked operation produces one history/trace entry.
11. A failed source preflight leaves no partial document or binding change.

12. A workspace fork contains no runtime identities shared with the source graph, except external immutable source references intentionally shared by value.
13. Internal alias relationships are preserved by graph copying.
14. A presentation action on an analysis-owned object carries its owner and never guesses from focus.
15. A destructive source reset is never presented as a preserving operation.

8.3 User-experience invariants

1. A visible linked indicator is not color-only.
2. The user can determine what will change before a conflicting group merge.
3. Switching a group updates all member views in one render transaction.
4. Unlinking does not visually change the selected analysis.
5. A full workspace duplicate does not change when the original is edited.
6. A mirror is named as a mirror.
7. A source mapping is never silently guessed when ambiguous.
8. Invalid candidate sources produce actionable diagnostics.
9. A template asks once per input slot, not once per tile.
10. A command initiated from a pipeline, encoding, or relation uses that object's owning analysis.

---

9. Recommended subject-and-port architecture

The current `documentBindingId` model can evolve cleanly into normalized typed subjects.

9.1 Normalize bindings

Proposed

```ts
export type BindingId = string;

export interface SubjectValueMap {
  analysis: {
    documentId: DocId | null;
  };
}

export type SubjectKind = keyof SubjectValueMap;

export type SubjectBinding = {
  [K in SubjectKind]: {
    id: BindingId;
    kind: K;
    value: SubjectValueMap[K];
    /**
     * Optional user-facing name such as “Regional population”.
     * The selected document name remains separate.
     */
    label?: string;
    /**
     * Monotonic revision for stale async preflight protection.
     */
    revision: number;
  };
}[SubjectKind];
```

The initial implementation has only one subject kind. The discriminated union still matters: it prevents an analysis port from accidentally attaching to a future row-selection binding.

The layout state gains:

```ts
// Proposed
interface LayoutState {
  views: Record<ViewId, AppView>;
  bindings: Record<BindingId, SubjectBinding>;
```

```
  // existing spaces, stages, pointers...
}
```

9.2 Replace document maps with named port bindings

Proposed

```
export interface AppView {
  id: ViewId;
  appId: AppId;

  /**
   * Port name → subject binding.
   *
   * "primary" replaces the current documents.primary role for ordinary
   * analysis-bound applications.
   */
  bindings: Record<string, BindingId>;

  title?: string;
}
```

The binding stores the selected document. The view stores only the connection.

This makes a group update one record:

```
binding.value.documentId = nextDocId;
binding.revision += 1;
```

All ports read through the same normalized record.

9.3 Declare ports in app descriptors

Proposed

```
export interface AppPortDescriptor<K extends SubjectKind = SubjectKind> {
  subject: K;
  label: string;
  required: boolean;
  access: "read" | "read-write";
}

export interface AppDescriptor {
  id: string;
  title: string;
```

```
  tone: string;
  ports?: Record<string, AppPortDescriptor>;
  duplicable: boolean;
  singleton: boolean;
  Component: ComponentType<AppProps>;
}
```

Examples:

```
// Proposed
registerApp({
  id: "chart",
  title: "chart",
  tone: "var(--pbui-tone-chart)",
  ports: {
    primary: {
      subject: "analysis",
      label: "Analysis",
      required: true,
      access: "read",
    },
  },
  duplicable: true,
  singleton: false,
  Component: ChartApp,
});
```

```
// Proposed
registerApp({
  id: "pipeline",
  title: "pipeline",
  tone: "var(--pbui-tone-pipeline)",
  ports: {
    primary: {
      subject: "analysis",
      label: "Analysis",
      required: true,
      access: "read-write",
    },
  },
  duplicable: true,
  singleton: false,
  Component: PipelineApp,
});
```

```ts
// Proposed future example
registerApp({
  id: "compare",
  title: "compare",
  ports: {
    left: {
      subject: "analysis",
      label: "Left analysis",
      required: true,
      access: "read",
    },
    right: {
      subject: "analysis",
      label: "Right analysis",
      required: true,
      access: "read",
    },
  },
  // ...
});
```

The current `docBound` helper becomes a derived query:

```ts
// Proposed
const analysisPorts = (app: AppDescriptor) =>
  Object.entries(app.ports ?? {}).filter(
    ([, port]) => port.subject === "analysis",
  );
```

9.4 A group is a derived set, not a container

A binding is not required to own a list of members. Membership is derived by scanning or indexing view ports that reference the binding.

This avoids two sources of truth:

```
binding.members says [A, B]
views say A, C
```

For performance, selectors can maintain a memoized reverse index:

```ts
// Proposed derived selector
Record<BindingId, Array<{ viewId: ViewId; port: string }>>
```

The persisted truth remains `view.bindings`.

9.5 Private selection is still a binding

A one-view group should not omit the binding record. A private binding is meaningful:

- it has identity for PBUI;
- it can be named;
- it can be rebound;
- it can later gain members;
- it can be forked;
- it simplifies invariants.

The current fallback from missing binding ID to view ID is useful only for migration.

9.6 Read and write access

Read versus read-write describes whether the application mutates the selected analysis, not whether the binding can be rebound.

A chart is read-only with respect to document content but may still expose a control that rebinds its `primary` port. A pipeline editor mutates the document and may rebind the same port.

A later permissions layer can combine:

```
port access
document ACL
workspace role
command-specific authorization
```

Do not encode all of that into the binding.

9.7 Keep view-local state local

The analysis binding should not contain:

- chart width and height;
- zoom and pan;
- hover cursor;
- selected row;
- table sorting;
- visible pipeline step;
- open editor section;
- local formatting preferences.

These values describe a presentation or interaction state, not the selected analytical composition.

Some may later become independent typed subjects. For example:

```
// Future, not part of the first migration
interface SubjectValueMap {
  analysis: { documentId: DocId | null };
  filterSet: { filters: FilterSpec[] };
  timeRange: { start: Instant; end: Instant };
  rowSelection: { relation: RelationIdentity; rowKeys: string[] };
}
```

Each app would opt into only the ports it supports.

9.8 Core reducer operations

Proposed

```
setBindingSubject({
  bindingId,
  expectedRevision?,
  value,
});

mergeBindings({
  sourceBindingId,
  targetBindingId,
  mergedBindingId,
});

detachViewPort({
  viewId,
  port,
  newBindingId,
});

attachViewPort({
  viewId,
  port,
  bindingId,
});

renameBinding({
  bindingId,
  label,
});

deleteBinding({
  bindingId,
});
```

`mergeBindings` should rewrite every port that references either old binding to the fresh merged binding. The source binding's value wins unless the command supplies an explicit winner.

Using a fresh ID rather than reusing the source ID has one benefit: stale commands that captured either old group can detect that the group no longer exists. Reusing the source ID has the benefit of fewer rewrites. Either is viable; consistency and revision checks matter more than the choice.

9.9 Migration from current `AppView`

A deterministic migration can preserve every current group.

For each current `AppView`:

1. compute the effective current group ID: `view.documentBindingId ?? view.id`;
2. for each document role in that group, create one analysis binding;
3. map the old role name to a port name;
4. set each member view's port to that binding;
5. verify all members agree on the document selected for that role;
6. if old data disagrees, choose a deterministic first member and record a migration diagnostic.

For the current four apps:

```
documents.primary → bindings.primary
```

The old role map allowed several document roles to share one `documentBindingId` as one unit. Normalized bindings allow each port to be linked independently, which is more expressive.

---

10. PBUI presentation vocabulary

Linking should become part of the application's semantic presentation vocabulary rather than remain tile-specific chrome.

10.1 New presentation types

Proposed

```
export interface AnalysisBindingRef {
  bindingId: BindingId;
  documentId: DocId | null;
  label: string;
  memberCount: number;
}

export interface AnalysisPortRef {
  viewId: ViewId;
  port: string;
  appId: AppId;
  bindingId: BindingId;
  subjectKind: "analysis";
  documentId: DocId | null;
  label: string;
  access: "read" | "read-write";
}

export interface PipelineRef {
  docId: DocId;
  terminal: RelationRef;
}

export interface EncodingRef {
  docId: DocId;
  viewId: GraphicViewId;
}

export interface AnalysisSourceRef {
  docId: DocId;
  sourceNodeId: SourceNodeId;
  source: SourceRef;
}

export interface OutputRelationRef {
  docId: DocId;
  relation: RelationRef;
}
```

```
export interface StepRef {
  docId: DocId;
  stepId: TransformId;
}
```

The Datalab `PresentationValues` gains:

```
// Proposed
interface PresentationValues {
  // existing...
  analysisBinding: AnalysisBindingRef;
  analysisPort: AnalysisPortRef;
  pipeline: PipelineRef;
  encoding: EncodingRef;
  analysisSource: AnalysisSourceRef;
  outputRelation: OutputRelationRef;
  step: StepRef; // replaces string
}
```

The existing `doc` presentation can remain the internal presentation of the owning analysis document. Product labels may say “Analysis.”

10.2 Identity policies

Analysis binding

```
identityDomain: "analysis-binding",
identity: (ref) => ref.bindingId,
```

Analysis port

```
identityDomain: "analysis-port",
identity: (ref) => JSON.stringify([ref.viewId, ref.port]),
```

The port identity does not change when it is rebound.

Pipeline

If a document has one current terminal pipeline:

```
identityDomain: "analysis-pipeline",
identity: (ref) => ref.docId,
```

If pipelines later become separately versioned artifacts, their own stable ID should replace this.

Encoding

```
identityDomain: "analysis-encoding",
identity: (ref) => JSON.stringify([ref.docId, ref.viewId]),
```

Source owned by an analysis

```
identityDomain: "analysis-source-node",
identity: (ref) => JSON.stringify([ref.docId, ref.sourceNodeId]),
```

Output relation

```
identityDomain: "analysis-relation",
identity: (ref) =>
  JSON.stringify([ref.docId, ref.relation.kind, relationLocalId(ref.relation)]),
```

Step

```
identityDomain: "analysis-step",
identity: (ref) => JSON.stringify([ref.docId, ref.stepId]),
```

Owner qualification prevents collisions between two documents that happen to use the same generated step ID.

10.3 Do not convert a catalog source directly to an analysis

A plain `SourceRef` from the source browser does not identify an owning analysis. One source can be used by zero, one, or many documents.

Therefore this conversion is invalid:

```
catalog source —X—> analysis document
```

The source may instead participate in a command that has two arguments:

```
fork-analysis-onto-source(binding, source)
```

An `AnalysisSourceRef`, by contrast, can convert to its owner:

```
analysisSource(doc α, node s) → doc α
```

10.4 Conversions to the owning analysis

Proposed

```
const conversions = [
  {
    id: "pipeline-to-analysis",
    from: "pipeline",
    to: "doc",
    cost: 1,
    convert: ({ value }) => ({
      type: "doc",
      value: value.docId,
    }),
  },
  {
    id: "encoding-to-analysis",
    from: "encoding",
    to: "doc",
    cost: 1,
    convert: ({ value }) => ({
      type: "doc",
      value: value.docId,
    }),
  },
  {
    id: "analysis-source-to-analysis",
    from: "analysisSource",
    to: "doc",
    cost: 1,
    convert: ({ value }) => ({
      type: "doc",
      value: value.docId,
    }),
  },
  {
    id: "output-relation-to-analysis",
    from: "outputRelation",
    to: "doc",
    cost: 1,
    convert: ({ value }) => ({
      type: "doc",
      value: value.docId,
    }),
  },
  {
    id: "step-to-analysis",
    from: "step",
    to: "doc",
```

```
    cost: 1,
    convert: ({ value }) => ({
      type: "doc",
      value: value.docId,
    }),
  },
];
```

These are interpretations, not subtype relationships. A pipeline is not an analysis document; it belongs to one.

10.5 Descriptor descriptions

Descriptions should explain ownership and link state.

Example:

```
Analysis port
    Chart · primary
    Linked group: Regional population
    Members: 5
    Selected analysis: Population by region
```

Example pipeline:

```
Pipeline
    Analysis: Population by region
    Source: lab/census v2/rows.csv
    Enabled transforms: 1
    Output: region, population_total
```

Descriptions are especially valuable in PBUI menus because they make commands understandable when the same object appears in several places.

---

11. PBUI commands, selectors, actions, and conversions

11.1 Link-port input context

The basic command accepts an `analysisPort`.

Proposed

```
const target = await pbui.accept({
  prompt: "LINK ANALYSIS — choose another analysis port",
  selector: presentationSelectors.type("analysisPort", {
    id: "compatible-unlinked-analysis-port",
    cache: "identity",

    prepare: ({ environment }) => {
      const source = environment.analysisPort(sourceViewId, sourcePort);
      const compatible = new Set(
        environment
          .analysisPorts()
          .filter((candidate) =>
            candidate.subjectKind === source.subjectKind &&
            candidate.bindingId !== source.bindingId &&
            !(candidate.viewId === source.viewId && candidate.port === source.port),
          )
          .map((candidate) =>
            `${candidate.viewId}:${candidate.port}`,
          ),
      );

      return (candidate) =>
        compatible.has(`${candidate.viewId}:${candidate.port}`);
    },
  }),
});
```

The prepared function builds a coherent candidate set once. It does not repeatedly traverse the entire layout during hit testing.

The reducer must revalidate the target at commit because the layout may have changed while the input context was open.

11.2 Rebind-group input context

A command can accept any presentation convertible to `doc`.

```
// Proposed
const result = await pbui.accept({
  prompt: `USE AN ANALYSIS IN ${bindingLabel}`,
  selector: {
    types: "doc",
    includeSubtypes: true,
    cache: "identity",
    where: (reference) =>
      reference.value !== currentDocumentId,
  },
});
```

PBUI conversion search allows the user to click:

- a document chip;
- a pipeline;
- an encoding;
- an output relation;
- an owner-qualified source;
- a step.

The result is the owning `doc`.

11.3 Selector preparation and performance

Selector `where` and `prepare` have different purposes.

Use `where` for a cheap local predicate:

```
where: (port) => port.subjectKind === "analysis"
```

Use `prepare` when eligibility requires:

- a workspace traversal;
- permission lookup;
- construction of a `Set`;
- schema compatibility index;
- exclusion of an entire binding group;
- capture of a coherent revision snapshot.

Do not execute a source query, compile a document, or evaluate rows from a presentation predicate. Expensive work belongs before the input context or in a command preflight after selection.

11.4 Action rules

Common actions should be supplied by selector-driven action rules rather than duplicated in every descriptor.

Proposed

```
{
  id: "inspect-analysis-link",
  selector: {
    types: ["analysisBinding", "analysisPort"],
  },
  priority: 50,
  actions: (reference) => ({
    id: "inspect-link",
    label: "Inspect link group",
    verb: {
      kind: "inspectAnalysisBinding",
      bindingId: reference.value.bindingId,
    },
  }),
}
```

```
{
  id: "rebind-owning-analysis",
  selector: {
    types: [
      "pipeline",
      "encoding",
      "analysisSource",
      "outputRelation",
      "step",
      "doc",
    ],
  },
  actions: (reference) => ({
    id: "use-in-linked-group",
    label: "Use owning analysis in linked group…",
    verb: {
      kind: "beginRebindAnalysisGroup",
      source: reference,
    },
  }),
}
```

The second verb begins a partial command: the clicked object supplies the analysis; the user still needs to choose the destination group. The inverse interaction is also useful: start from a group and choose the analysis.

11.5 Serializable verbs and an application command host

PBUI descriptors currently return serializable verbs. That is valuable for:

- testing;
- replay;
- remote transport;
- trace logging;
- undo;
- command palettes.

An action that requires a second presentation argument should not store a JavaScript closure in a menu item.

Proposed command-host flow

```
descriptor action
    ↓
serializable begin-command verb
    ↓
PbuiCommandHost
    ↓
pbui.accept for missing argument
    ↓
completed serializable domain verb
    ↓
store transaction
```

Example verbs:

```ts
// Proposed
type LinkVerb =
  | {
      kind: "beginLinkAnalysisPort";
      sourceViewId: ViewId;
      sourcePort: string;
    }
  | {
      kind: "linkAnalysisPorts";
      source: PortAddress;
      target: PortAddress;
    }
```

```
  | {
      kind: "beginUseAnalysisInBinding";
      documentId: DocId;
    }
  | {
      kind: "setAnalysisBindingDocument";
      bindingId: BindingId;
      documentId: DocId;
      expectedRevision: number;
    };
```

This is the PBUI equivalent of a command with missing arguments.

11.6 “Add all compatible views” command

For the common workspace setup, selecting every port individually is unnecessary.

A command can:

1. start from a binding or port;
2. derive the current workspace;
3. collect all unlinked compatible `primary` analysis ports;
4. preview the members;
5. attach them atomically.

This command should exclude:

- ports already in the group;
- ports with another subject kind;
- optional ports the app has not enabled;
- ports in other workspaces unless explicitly requested;
- read/write conflicts only if a permission policy requires it.

Prepared selectors are still useful when the user wants to pick a subset manually.

11.7 Conflict preview on merge

If source and target bindings select different analyses, the operation changes the target group’s visible content.

The UI should preview:

```
Link “Chart · primary” to “Pipeline · primary”?

The pipeline group currently uses “Climate readings”.
The chart group uses “Population by region”.
```

```
Source wins:
3 views will switch to “Population by region”.
```

Options:

- **Link and use Population by region**
- **Link and use Climate readings**
- **Cancel**

For same-document groups, linking can be immediate.

The low-level reducer should accept an explicit winner so the confirmation decision is part of the command payload rather than hidden UI state.

---

12. Interaction design

12.1 Replace `DocBar` with an analysis-aware port bar

The current `DocBar` can evolve into `AnalysisBar`.

A compact form:

```
Analysis  [Population by region ▾]  [⛓ Regional population · 5]  [+]
```

The elements are:

- label: `Analysis`;
- document/analysis chip;
- selector;
- link-group chip with chain icon, optional name, and member count;
- create/fork menu.

A private binding may show:

```
[⛓ 1]
```

or an outline chain. A linked binding may show:

```
[⛓ Regional population · 5]
```

The chain state must not rely only on color.

12.2 Present the port, not merely the tile title

The group chip is an `analysisPort` presentation. This provides a precise target and remains correct for multi-port applications.

The tile title remains a `<tile>` presentation for layout actions such as duplicate, close, swap, and inspect.

These are different objects with different menus.

12.3 Chain-menu commands

A practical menu:

```
Analysis link
──────────────────────────────
Link to another view…
```

```
Add all compatible views here
Use another visible analysis…
──────────────────────────────────
Unlink this view
Manage group…
Rename group…
──────────────────────────────────
Fork analysis…
Fork analysis onto source…
```

The exact menu should vary by state:

- hide `Unlink` for one-member bindings;
- hide `Add all` if no eligible ports exist;
- disable `Fork onto source` until a source is selected or begin a source accept command;
- show member count and current selected analysis in the header.

12.4 Manage-group panel

For more than a few views, a small inspector is useful:

```
Regional population
Analysis: Population by region
Members: 5

✓ Dataset · primary
✓ Pipeline · primary
✓ Table · primary
✓ Encoding · primary
✓ Chart · primary

[Add visible view] [Detach selected] [Rename] [Fork]
```

Every member row should itself be a presentation, making inspection and navigation compositional.

12.5 Visual feedback during accept

While linking:

- eligible analysis-port chips receive a strong focus/highlight treatment;
- ineligible ports remain visible but are not interactive;
- the prompt names the source group;
- Escape cancels;
- Enter/Space activates a focused presentation;
- the source port remains marked;

- all occurrences of the same semantically eligible object may be highlighted consistently.

Do not highlight the entire tile when only one port is eligible.

12.6 Switching from a dropdown

The analysis selector in any linked member writes the binding's value. It should not dispatch a per-view change.

A selection label can clarify scope:

```
Change analysis for 5 linked views
```

For a private binding:

```
Change analysis for this view
```

12.7 Switching from an object menu

A visible pipeline can offer:

```
Use this analysis in…
    Regional population
    Comparison · left
    Comparison · right
```

For many groups, selecting "Use this analysis in…" should start a PBUI accept context for an `analysisBinding` presentation rather than render a long nested menu.

12.8 Drag and drop

Drag-and-drop can be added later as another gesture over the same commands:

- drag an analysis chip onto an analysis-port chip to rebind;
- drag one chain chip onto another to merge groups;
- hold a modifier to fork rather than share.

The command semantics must exist independently first. Dragging is a gesture, not the data model.

12.9 Naming groups

Binding labels are optional. Default display can derive from:

1. explicit binding label;
2. selected document name;

3. source name;
4. `Analysis link`.

Names are especially useful in templates and commands:

```
Main analysis
Baseline
Candidate
Regional population
```

Renaming a binding does not rename the document.

---

13. Switching to another existing analysis

This is the simplest and safest operation.

13.1 Semantic meaning

When the user selects a new pipeline “defined on another dataset,” the current model does not have a standalone pipeline object separate from its document. The correct meaning is:

> Select the `GraphicDocument` that owns that pipeline.

That document brings its own source, transforms, result relation, encoding, and plot specification.

13.2 Command sequence

```
user selects pipeline β
    ↓
pipeline presentation converts to doc β
    ↓
command chooses binding A
    ↓
setBindingSubject(A, { documentId: β })
    ↓
chart/table/pipeline/encoding/dataset members render β
```

No document content is copied or mutated.

13.3 Atomic update

The normalized binding update is one state change. Every member selector observes the same binding record during the same Redux commit.

There should be no intermediate state in which the chart uses β but the table still uses α.

13.4 Effects and execution

After the rebind:

- the chart requests the plot for β;
- the table requests the result for β;
- the pipeline editor reads β’s transforms;
- the encoding editor reads β’s root view;
- the dataset app reads β’s root source.

`AnalysisCoordinator` should coalesce shared execution requests by semantic key.

13.5 Mutation from a linked editor

If the user edits the pipeline in one linked pipeline view, they are editing the selected document itself. Every other member observes the resulting document revision.

This is stronger than sharing only selection. It is intentional: the selected analysis composition is one application object.

To experiment independently, the user should choose **Fork analysis**, which creates a new document and rebinds the group to it.

13.6 Two groups may select the same document without being linked

This distinction is essential:

```
binding A → document α
binding B → document α
```

The two groups currently show the same analysis, and edits to α are visible in both because the document object is shared. But changing binding A to β does not change binding B.

To make the analytical content independent as well, fork the document.

This yields three useful sharing levels:

Relationship	Same view?	Same binding?	Same document?
linked placement	yes	yes	yes
linked analysis views	no	yes	yes
independent selectors on same analysis	no	no	yes
fully forked analysis	no	no	no

The UI should not collapse these distinctions.

---

14. Retargeting or forking an analysis onto another dataset

The reusable-workspace goal requires preserving a setup across sources. This is a compilation problem, not just a binding update.

14.1 Keep three commands distinct

Open source as new analysis

Creates a default document from the source.

```
source S → new default document γ
```

No pipeline or encoding is preserved.

Replace source and reset setup

Uses the current `setDocSource` behavior.

```
document α(source S, pipeline P, encoding E)
    ↓
document α(source T, empty pipeline, default encoding)
```

This is destructive but predictable.

Fork analysis onto source

Recommended for reusable workspaces.

```
document α(source S, pipeline P, encoding E)
    ↓ clone and adapt
document β(source T, pipeline P′, encoding E′)
    ↓
binding A: α → β
```

The original document remains unchanged.

14.2 Why fork should be the default

Retargeting the existing document changes every group that independently selects that document, even when those groups are not linked.

Forking limits the change to the chosen binding group after it is rebound.

It also provides:

- a clear undo boundary;
- a safe place for field mapping;
- no partial mutation on validation failure;
- provenance from old document to new;
- a natural comparison between original and adapted analysis.

An explicit “retarget in place” command can exist for advanced users.

14.3 Preflight pipeline

A robust fork-on-source operation should proceed as follows.

Phase 1: capture

1. Resolve the source analysis binding.
2. Resolve its current document.
3. Capture the binding revision and document semantic revision.
4. Resolve the candidate `SourceRef`.
5. Fetch or retrieve the target source schema.

Phase 2: clone

6. Deep-clone the current `GraphicDocument`.
7. Mint a fresh document ID and user-visible name.
8. Preserve source-node identity inside the clone where possible.
9. Replace the root source value without clearing transforms or views.
10. Adapt source scope and row limit according to dataset/stream semantics.

Phase 3: analyze compatibility

11. Compile the old document against its current schema to obtain resolved field provenance.
12. Derive the set of source-origin field requirements.
13. Match required fields against the candidate schema.
14. Classify the candidate:
    - exact;
    - compatible with deterministic coercion;
    - mapping required;
    - incompatible.
15. Request explicit mappings for unresolved or ambiguous fields.

Phase 4: rewrite

16. Rewrite only source-origin authoring field references.
17. Preserve generated transform-output names unless the user explicitly changes them.
18. Update labels or metadata recording the mapping.
19. Compile the candidate document against the candidate schema.
20. Refuse commit if compilation emits errors.

Phase 5: commit

21. Verify captured binding and document revisions are still current.
22. Dispatch one transaction:
    - add the candidate document;
    - rebind the chosen analysis binding;
    - append one trace/history record.
23. Focus or announce the updated group.
24. Provide one-step undo.

14.4 Pure helper boundary

The transformation should be implemented as a pure model function before store integration.

Proposed

```
export interface RetargetPlan {
  sourceDocumentId: DocId;
  targetSource: SourceRef;
  targetSchema: SourceSchema;
  mappings: FieldMapping[];
  mode: "fork" | "in-place";
  newDocumentId?: DocId;
  newName?: string;
}

export type RetargetResult =
  | {
      ok: true;
      document: GraphicDocument;
      diagnostics: Diagnostic[];
      contract: AnalysisContract;
    }
  | {
      ok: false;
      reason:
        | "mapping-required"
        | "incompatible"
```

```
        | "compile-error";
      diagnostics: Diagnostic[];
      requirements: FieldRequirement[];
    };

export function retargetGraphicDocument(
  document: GraphicDocument,
  plan: RetargetPlan,
): RetargetResult;
```

The function should not dispatch, query the network, read a clock, or mint random IDs internally. Inputs supply all nondeterministic values.

14.5 Preserve the source node ID

Current `replaceDocumentSource` already preserves the first source node's ID when resetting the document. The preserving retarget operation should do the same.

This matters because source relation references and resolved field IDs may derive from source-node identity. Stable internal source-node identity reduces unnecessary rewrite.

It does not eliminate the need to validate field names and types.

14.6 Preserve derived output aliases

Suppose the old pipeline contains:

```
group by region
sum population as population_total
```

The new source maps:

```
region → city
population → residents
```

The adapted transform should be:

```
group by city
sum residents as population_total
```

The aggregate output alias remains `population_total`. Downstream encoding can continue to use the same derived field:

```
x → city
```

```
y → population_total
```

Only the source-origin dimension reference changes from `region` to `city`. A global string replacement would be unsafe because the same text can occur in labels, derived aliases, filter constants, or metadata.

14.7 Use provenance, not text search

The compiler already resolves fields and records provenance for operation outputs. The retargeter should use contextual resolved information to classify references:

- source-origin field;
- transform-generated field;
- parameter;
- unresolved field;
- literal string unrelated to a field.

Rewriting must traverse typed authoring structures:

- transform group-by fields;
- aggregate measure inputs;
- filter field references;
- sort keys;
- view encodings;
- analysis settings that contain field refs;
- facet fields;
- scale domains if field-based;
- any future transform-specific field positions.

A central visitor over `AuthoringFieldRef` locations is safer than independent ad hoc rewriting in each command.

14.8 Do not execute rows for compatibility checks

Compatibility is primarily a schema and compile question.

The current frontend can request large tables, with a high row budget in some paths. A target-source picker must not fetch up to a million rows merely to decide whether names and types match.

Add a lightweight schema endpoint or cache:

```
SourceRef → schema fingerprint + fields
```

Run row execution only after the candidate compiles and becomes selected.

14.9 Streams require additional policy

Retargeting from a bounded dataset to a stream may have valid schema compatibility but different execution semantics.

The preflight should account for:

- bounded versus unbounded input;
- ordering requirements;
- event-time fields;
- window requirements for aggregates;
- source-specific limits;
- unavailable historical rows;
- schema drift.

The first version may explicitly restrict preserving retarget to sources with the same source kind. The type model should leave room for a later cross-kind policy.

14.10 Diagnostics

A failed adaptation should explain use sites:

```
Cannot use “city-demographics” with “Population by region”.

Required field “region” is missing.
Used by:
  • aggregate-population.groupBy[0]
  • root-view.encodings.x
  • root-view.encodings.color

Possible nominal fields:
  • city
  • county
```

This is more actionable than “chart failed to compile.”

---

15. Schema contracts and field mapping

A reusable analysis implicitly defines a contract over its input source. Making that contract explicit improves retargeting and templates.

15.1 Field requirements

Proposed

```ts
export interface FieldRequirement {
  id: string;
  originalName: string;
  semanticType: FieldType;
  nullable: "allowed" | "forbidden" | "unknown";
  role:
    | "dimension"
    | "measure"
    | "time"
    | "identifier"
    | "weight"
    | "other";
  useSites: FieldUseSite[];
}

export interface FieldUseSite {
  owner:
    | { kind: "transform"; transformId: TransformId }
    | { kind: "view"; viewId: GraphicViewId };
  path: string;
}
```

The role can initially be inferred from use:

- aggregate group key → dimension;
- aggregate numeric input → measure;
- temporal encoding → time;
- join key → identifier.

Users or templates can later name roles explicitly.

15.2 Analysis contract

Proposed

```ts
export interface AnalysisContract {
  version: 1;
  sourceKinds: Array<"dataset" | "stream">;
  fields: FieldRequirement[];
  /**
   * Fingerprint of the original source schema for exact-match shortcuts.
   */
  sourceSchemaFingerprint?: string;
}
```

The contract belongs to a template or can be derived on demand from a document. It should not duplicate the entire compiled plan.

15.3 Compatibility classes

Exact

Every requirement resolves by stable name and compatible type.

No prompt is required.

Compatible with deterministic coercion

Examples depend on product policy:

- integer to quantitative number;
- non-nullable to nullable requirement;
- date string with declared parse rule to temporal.

Coercions must be explicit in the plan and surfaced to the user.

Mapping required

At least one field is absent by name but one or more candidate fields have compatible type/role.

The user must choose.

Incompatible

No candidate can satisfy a required field or a transform capability is unavailable.

The system should not create a broken document.

15.4 Candidate ranking

Suggestions may be ranked by:

1. exact name;
2. case-insensitive normalized name;
3. stored role mapping from the template;
4. semantic type;
5. physical type;
6. profile similarity;
7. name similarity;
8. distinct-count characteristics;
9. unit metadata.

Ranking is assistance, not authorization to silently map ambiguous fields.

15.5 Mapping UI

A concise mapping table:

Analysis role	Existing field	Required type	Target field	Status
group dimension	`region`	nominal	`city`	selected
measure	`population`	quantitative	`residents`	selected
tooltip	`area_km2`	quantitative	`land_area`	suggested

The preview should show affected pipeline and encoding locations.

Buttons:

- **Create adapted analysis**
- **Back**
- **Use default analysis instead**
- **Cancel**

15.6 Stable semantic roles in templates

Field names are incidental. Template authors should be able to expose roles:

```
Category field
Population measure
Optional area measure
```

A template slot can save a mapping from role to blueprint field. At instantiation the user maps target fields to roles.

This makes templates portable across organizations with different naming conventions.

15.7 Schema drift after binding

A source version may change after the workspace was saved.

The analysis resource should compare the current schema fingerprint against the last validated fingerprint. On drift:

- do not silently reset;
- show a diagnostic badge on every linked analysis port;
- compile again;
- preserve the group;
- offer repair mapping;
- keep the last valid result visible if product policy permits, clearly marked stale.

---

16. A linked dataset application

The current source catalog and the requested linked dataset view serve different purposes.

16.1 Keep `SourceApp` as a catalog

The singleton source browser answers:

- What drops exist?
- What datasets and streams are available?
- Which versions and files exist?
- What source can I open or apply?

It should remain global and ownerless.

16.2 Add `DatasetApp` or `AnalysisSourceApp`

The new application is analysis-bound.

Proposed descriptor

```
registerApp({
  id: "dataset",
  title: "dataset",
  tone: "var(--pbui-tone-source)",
  ports: {
    primary: {
      subject: "analysis",
      label: "Analysis",
      required: true,
      access: "read-write",
    },
  },
  duplicable: true,
  singleton: false,
  Component: DatasetApp,
});
```

It reads the selected document from the binding, then obtains the document's root source.

16.3 Dataset application contents

A useful panel shows:

```
Analysis: Population by region
Source: lab / census / v2 / rows.csv
Kind: dataset
Rows: 24
Fields: 4
Limit: 2,000
Schema fingerprint: …
Compatibility contract: 2 required fields, 1 optional

Required by this analysis
  region       nominal       group, x, color
  population   quantitative  aggregate measure

[Browse sources] [Fork onto source…] [Replace and reset…]
```

The source identifier is an `analysisSource` presentation because it carries both the source value and owning `docId`.

Fields shown here should be owner-qualified `field` presentations.

16.4 Source-browser actions

A catalog `<source>` presentation can offer:

```
Open as new analysis
Fork linked analysis onto this source…
Replace source and reset…
Inspect schema
Copy source reference
```

The second action needs an analysis binding argument. It can:

- use the currently focused analysis port only when that focus is explicit;
- present a list of visible binding chips;
- or start a PBUI accept context for `analysisBinding`.

It should not target `activeDocId` invisibly.

16.5 Source selection from a linked group

The inverse command starts from a binding:

```
Fork “Regional population” onto source…
```

Then PBUI accepts a `<source>` presentation from anywhere visible. Because source compatibility may require asynchronous schema work, the input context should initially test only cheap eligibility such as source kind and permission. Full preflight follows selection.

---

17. Workspace mirror, duplicate, fork, and template instance

Workspace copying must state exactly which identities are shared.

17.1 Four useful operations

Mirror workspace

Current `cloneSpace` semantics.

- new workspace ID;
- new layout node IDs;
- same logical view IDs;
- same binding IDs;
- same document IDs.

Use case: another arrangement or stage view over exactly the same running applications.

Suggested label: **Mirror workspace**.

Duplicate layout with independent selectors

- new workspace and layout IDs;
- new logical view IDs;
- new binding IDs preserving internal group topology;
- same document IDs.

Use case: begin with the same analyses but allow each workspace to select different documents later. Edits to a shared document still appear in both.

Suggested advanced label: **Duplicate views** or **Duplicate layout**.

Fork workspace

- new workspace and layout IDs;
- new logical view IDs;
- new binding IDs;
- new document IDs;
- internal aliases preserved;
- no aliases back to the original.

Use case: apply the same analytical setup to another source without changing the original.

Suggested ordinary user label: **Duplicate workspace**.

Instantiate template

- new runtime IDs;
- layout and group topology from template;
- each named slot bound to an existing analysis or to a newly adapted blueprint document.

Use case: reusable workspaces dedicated to a task.

17.2 Identity matrix

Operation	Layout nodes	Logical views	Bindings	Documents
Mirror	new	shared	shared	shared
Duplicate layout	new	new	new	shared
Fork workspace	new	new	new	new
Template instance	new	new	new	selected or new

Naming these operations prevents surprising behavior.

17.3 Full graph-fork algorithm

Proposed

1. Traverse the target workspace tree.
2. Collect unique reachable `viewId`s.
3. Build `oldViewId → newViewId`.
4. Collect the bindings referenced by those views.
5. Build `oldBindingId → newBindingId`.
6. Collect documents referenced by those bindings.
7. Build `oldDocId → newDocId`.
8. Clone each document once and rewrite its envelope identity.
9. Clone each binding once and rewrite document references.
10. Clone each view once and rewrite binding references.
11. Clone the tree, rewriting leaf view IDs and minting node IDs.
12. Commit documents, bindings, views, and workspace in one transaction.
13. Verify no cloned runtime ID equals its source ID.
14. Verify alias-equivalence classes are preserved.

17.4 Preserve multiple placements correctly

If two leaves in the source tree reference one logical view, the forked leaves must reference one new logical view, not two.

Likewise, if five source views reference one binding, the five cloned views must reference one new binding.

Likewise, if two bindings intentionally select one document, the clone policy must decide whether they select one new cloned document or separate cloned documents. For a full graph fork, preserving object aliasing means one new cloned document.

17.5 Reuse portable bundle machinery

The existing `DocCollector`, `ViewCollector`, portable indices, and hydration code already solve most of these alias rules.

A practical first implementation can perform an in-memory bundle round trip:

```
// Proposed conceptual implementation
const bundle = bundleForWorkspace(state, spaceId, exportedAt);
const hydrated = hydrateWorkspaceBundle(bundle, idFactory);
dispatch(applyHydratedWorkspace(hydrated));
```

A later refactor can extract shared `collectWorkspaceGraph` and `cloneWorkspaceGraph` functions to avoid serialization overhead.

The important point is to reuse tested graph semantics rather than duplicate them in `layout.ts`.

17.6 Boundary of a workspace-local fork

A binding group may have members in several workspaces.

Forking one workspace should normally copy only ports reachable from that workspace. The cloned ports form their own new binding. External members remain linked to the original group.

This is predictable:

```
original binding A
  workspace 1: chart, pipeline
  workspace 2: table

fork workspace 1
  new binding B: cloned chart, cloned pipeline
  binding A: original chart, original pipeline, original table
```

The UI may warn:

```
This workspace shares an analysis link with 1 view outside it.
The duplicate will contain an independent copy of the members visible here.
```

17.7 Fork then retarget

The desired user flow becomes:

1. **Duplicate workspace** — full fork.
2. In the copy, choose **Fork analysis onto source…** or, because the documents are already independent, **Retarget copied analysis onto source…**.
3. Map fields if needed.
4. The copied linked group updates.
5. The original remains unchanged.

A combined command can optimize this:

```
Duplicate workspace for another dataset…
```

It performs workspace fork followed by source preflight for one or more exposed analysis slots.

---

18. Parameterized workspace templates

A reusable workspace is best represented as a graph with named input slots.

18.1 Snapshot versus template

A snapshot says:

```
Open these exact documents in this exact layout.
```

A parameterized template says:

```
Open this layout and link topology.
The group named “Main analysis” needs an analysis.
Use this document as a blueprint when the user supplies a source.
```

Both are useful. They should be named differently or stored with an explicit mode.

18.2 Template slot model

Proposed

```ts
export interface WorkspaceTemplateSlot {
  id: string;
  kind: "analysis";
  label: string;
  description?: string;

  /**
   * Optional document in the portable document array used as a blueprint.
   */
  blueprintDocument?: number;

  /**
   * Optional default concrete analysis for snapshot-like behavior.
   */
  defaultDocument?: number;

  contract?: PortableAnalysisContract;

  required: boolean;
}
```

Portable binding entries can refer to either a document or a slot.

Proposed

```
export type PortableAnalysisSubject =
  | { kind: "document"; document: number }
  | { kind: "slot"; slot: string };

export interface PortableBinding {
  kind: "analysis";
  label?: string;
  subject: PortableAnalysisSubject;
}
```

Portable views refer to portable binding indices.

18.3 One linked group becomes one slot

For the population workspace:

```
slot: main-analysis
label: Main analysis

members:
  dataset.primary
  pipeline.primary
  table.primary
  encoding.primary
  chart.primary
```

Instantiation asks once for `Main analysis`.

A comparison template can expose two slots:

```
Baseline
Candidate
```

18.4 Instantiation choices

For each analysis slot, the user may:

Bind an existing analysis

No document clone is required unless the template policy says to fork it.

Select a source

Clone the blueprint document, adapt it to the selected source, and bind the new document.

Use the template default

Instantiate the stored document as a fresh clone.

Leave unbound

Only for optional slots. Member views show an explicit empty state and an analysis-port presentation that accepts a binding.

18.5 Save-as-template flow

When saving a workspace:

1. identify each distinct analysis binding reachable from the workspace;
2. show one row per group;
3. ask whether to:
    - embed as fixed analysis;
    - expose as slot;
    - omit;
4. let the user name exposed slots;
5. derive and preview contracts;
6. optionally strip source references from blueprints;
7. export the portable graph.

This converts the actual link topology into a reusable interface.

18.6 Privacy and portability

Templates should not embed:

- access tokens;
- session credentials;
- secret headers;
- local absolute paths unless explicitly allowed;
- data rows merely for preview;
- inaccessible private source references without a warning.

The existing credential-shaped-key audit remains useful. Slot-based templates can be more portable because they need not store the original source at all.

18.7 Template versioning

A template should record:

- format version;

- blueprint document version;
- contract version;
- required application IDs and versions;
- creation metadata;
- optional migration notes.

Instantiation should fail with a precise diagnostic when an app or transform type is unavailable.

---

19. Persistence and remote protocol

19.1 Local persistence

The normalized model requires a persistence version bump and a real migration.

The current persistence path should not merely reject every older version. A migration from current `AppView.documents` and `documentBindingId` is straightforward and valuable.

Proposed persisted layout

```
interface PersistedLayoutV5 {
  version: 5;
  views: Record<ViewId, {
    id: ViewId;
    appId: AppId;
    bindings: Record<string, BindingId>;
    title?: string;
  }>;
  bindings: Record<BindingId, SubjectBinding>;
  // spaces and stages...
}
```

Validation must ensure:

- binding IDs are unique;
- view references resolve;
- subject values resolve;
- app port kinds match;
- no impossible revisions;
- legacy roles are migrated deterministically.

19.2 Portable bundles

The current numeric `PortableView.binding` correctly preserves one equivalence relation. A generalized format should make bindings explicit because subjects may later have different kinds and named slots.

Proposed portable shape

```
interface WorkspacePayloadV4 {
  name: string;
  tree: PortableNode;
  views: PortableViewV4[];
  bindings: PortableBinding[];
  docs: PortableDoc[];
```

```
  slots?: WorkspaceTemplateSlot[];
  apps?: string[];
}

interface PortableViewV4 {
  app: string;
  title?: string;
  bindings: Record<string, number>;
}
```

Dense indices remain preferable to runtime IDs:

- no accidental cross-import identity;
- easy validation;
- small payload;
- preserved internal sharing.

19.3 Backward compatibility

Importer behavior:

- v3 `view.binding` + `documents` becomes one or more analysis bindings;
- missing binding index creates a private binding;
- old view document role `primary` becomes port `primary`;
- unsupported app ports produce a diagnostic rather than silent loss;
- old exact templates instantiate as snapshot templates.

Exporter emits only the current version.

19.4 Remote protocol

A durable protocol should normalize bindings instead of adding only another copied `document_binding_id` string.

Proposed protobuf direction

```
message SubjectBinding {
  string id = 1;
  string kind = 2;
  uint64 revision = 3;
  optional string label = 4;

  oneof value {
    AnalysisSubject analysis = 10;
  }
}
```

```
message AnalysisSubject {
  optional string document_id = 1;
}

message AppView {
  string id = 1;
  string app_id = 2;
  map<string, string> bindings = 3; // port name → binding id
  optional string title = 4;
}

message WorkbenchDocument {
  map<string, DocumentPayload> documents = 8;
  map<string, AppView> views = 9;
  map<string, SubjectBinding> subject_bindings = 10;
  // layout/stage fields...
}
```

The exact field numbers must be chosen against the existing schema and reserved-field policy.

19.5 Remote mutations

Useful domain mutations include:

```
put_subject_binding
delete_subject_binding
set_subject_value
attach_view_port
detach_view_port
merge_subject_bindings
put_document
```

A source-fork transaction needs atomic multi-entity application. Either:

- the protocol supports a transaction envelope;
- or the server exposes a high-level `fork_analysis_onto_source` command;
- or mutations carry one transaction ID and are committed together.

Sending “put document” and “rebind” independently can leave orphaned documents or dangling bindings after a partial failure.

19.6 Revision checks

Remote subject updates should include expected revision:

```
message SetAnalysisSubject {
  string binding_id = 1;
  optional string document_id = 2;
  uint64 expected_revision = 3;
}
```

The server rejects stale updates with current state. The client can refresh and show a conflict.

19.7 Document deletion

Deletion must be binding-aware.

Possible policy:

```
Delete analysis “Population by region”?
Referenced by:
  • Regional population · 5 views
  • Dashboard summary · 2 views

Choose replacement:
  [another analysis]
  [leave groups unbound]
  [cancel]
```

A force-delete mutation should include the chosen repair policy.

---

20. Performance design

Linked workspaces may put several views of one document on screen. The architecture should exploit shared semantics.

20.1 Normalized binding update cost

Current propagation scans all logical views and copies document maps.

Normalized update:

```
O(1) binding mutation
+ normal selector invalidation for actual consumers
```

A derived reverse membership index supports member count and management without changing persisted truth.

20.2 PBUI selector cost

Use prepared selectors to perform workspace traversal once per input context.

Good:

```
prepare:
    build Set<PortIdentity>
matches:
    Set.has(identity)
```

Bad:

```
matches:
    scan all workspaces
    resolve app descriptors
    query permissions
    compile schema
```

Identity caching is operation-scoped, so it cannot remain stale indefinitely.

20.3 Final validation at commit

Prepared predicates capture a snapshot. The selected port can disappear, change kind, join the source group, or lose permission while the user is choosing.

The reducer or command service must revalidate:

- source and target exist;
- port declarations still match;
- binding IDs still match expected revisions;
- permissions still hold.

The selector is for responsive interaction. The command is the authority.

20.4 Central per-document analysis resource

Several apps currently call analysis hooks independently. Create a shared resource keyed by document semantic identity/revision:

```
// Proposed conceptual API
const resource = analysisResources.forDocument(docId);

resource.compiled;
resource.outputTable;
resource.plot(width, height);
resource.schemaAt(relation);
resource.diagnostics;
resource.loading;
```

Responsibilities:

- compile once per semantic revision;
- execute once per logical relation/limit key;
- share output among table and chart;
- expose cheap schema-only results to pipeline and encoding editors;
- initialize default encodings once, not once per mounted view;
- preserve stale-result rejection.

20.5 Existing coordinator reuse

`AnalysisCoordinator` already has in-flight coalescing and semantic keys. The resource layer should use it rather than replace it.

The missing layer is stable document-level ownership of compile and initialization work.

20.6 Compatibility cache

Cache source preflight by:

```
(document semantic revision, target schema fingerprint, contract version)
```

Do not key only by document ID because the pipeline may change.

Cache results:

- exact;
- deterministic mapping;
- unresolved requirements;
- diagnostics.

Never cache authorization decisions longer than their policy permits.

20.7 Schema-only source access

Add or expose a query that returns:

- source identity/version;
- schema fingerprint;
- field names and semantic/physical types;
- nullability/profile summaries needed for ranking.

Do not materialize full rows for a source picker.

20.8 Avoid render-path evaluation

The current PBUI environment already distinguishes cheap `fieldsFor` from expensive `tableFor`, with comments guarding render paths.

Linked-port chips, descriptions, and hover states should use schema and metadata only. Full table evaluation belongs in explicit commands or analysis resources.

20.9 Relation-field inference

`fieldsAtRelation` and pipeline schema inference can be memoized by:

```
document semantic revision + relation identity
```

A pipeline panel should not recompute the same relation schema for every row and every linked view.

20.10 Large workspaces

For hundreds of views:

- maintain memoized binding membership indices;
- scope “Add all compatible” to the current workspace by default;
- virtualize large group managers;
- keep presentation identity keys compact;
- avoid embedding complete document objects in `AnalysisPortRef`;

- store only IDs and resolved display metadata.

---

21. Transactions, undo, and concurrency

21.1 One conceptual operation, one transaction

These should each be atomic:

- merge link groups;
- detach a port;
- rebind a group;
- fork an analysis;
- adapt an analysis onto a source;
- fork a workspace;
- instantiate a template.

A user should never see half the group updated.

21.2 High-level trace records

Examples:

```
analysis_group_linked
    sourceBinding
    targetBinding
    mergedBinding
    winnerDocument
    memberCount

analysis_group_rebound
    binding
    oldDocument
    newDocument

analysis_forked_to_source
    binding
    sourceDocument
    newDocument
    targetSource
    mappings

workspace_forked
    sourceWorkspace
    newWorkspace
    viewCount
    bindingCount
    documentCount
```

Do not log one low-level entry per port rewrite.

21.3 Undo

Useful inverse operations:

Rebind

Store the old document ID and expected current revision.

Detach

Store the old binding ID and fresh binding ID. Undo reattaches the port if neither has changed incompatibly.

Merge

Undo is more complex because members from two groups were combined. The command must record the original partition and values.

Fork analysis onto source

Undo rebinds to the old document. The new document can be retained as an orphaned recent analysis or garbage-collected when unreferenced, according to product policy.

Fork workspace

Undo removes the newly created workspace and any documents, bindings, and views reachable only from it.

A toast with **Undo** is a practical first UI.

21.4 Async preflight races

A source schema request and compile can take long enough for state to change.

Capture:

```
binding ID
binding revision
source document ID
source document semantic revision
target source version/fingerprint
```

Before commit, verify all still match. If not:

```
“The analysis changed while compatibility was being checked. Review the updated setup and try again.”
```

Do not commit a plan computed for stale transforms.

21.5 Concurrent group merges

Two clients may concurrently merge or rebind the same groups.

Expected-revision checks make the conflict visible. Server-side command application should be idempotent by command ID.

A merge command should reference the source and target binding revisions. If either is stale, reject rather than silently merging a different membership set.

21.6 Immutable snapshots for selectors

PBUI `prepare` should capture IDs and scalar metadata, not mutable object references that Immer may later replace.

The final command resolves current records by ID.

---

22. Accessibility and discoverability

22.1 Chain icon semantics

The glyph alone is insufficient. Use accessible labels:

- `Analysis link: private`
- `Analysis link: Regional population, 5 views`
- `Link this analysis port`
- `Unlink this view from Regional population`

The linked state needs text or count, not only color.

22.2 Keyboard flow

A complete keyboard interaction:

1. Focus the analysis-link button.
2. Press Enter.
3. The command prompt announces: `Choose another analysis port. Five eligible targets.`
4. Tab or arrow navigation moves among eligible presentations.
5. Enter selects.
6. Escape cancels and returns focus to the initiating control.
7. A status region announces the result.

22.3 Screen-reader relationship descriptions

An analysis port can expose:

```
Chart analysis. Population by region.
Linked with Dataset, Pipeline, Table, and Encoding.
```

The manage-group panel should use a real list with buttons, not a visually arranged set of spans.

22.4 Conflict dialogs

A conflict preview should identify:

- source group;
- target group;
- current analyses;
- number of affected views;
- winner choice.

Buttons must name the outcome, not `OK` and `Cancel` only.

22.5 Empty and invalid states

Unbound:

```
No analysis selected.
[Choose visible analysis] [Create analysis]
```

Schema-invalid after drift:

```
Analysis needs repair.
2 source fields no longer resolve.
[Review mappings]
```

The chain remains intact in both states. Errors should not silently detach views.

22.6 Tutorial affordances

A newcomer tutorial can use presentations:

```
1. Activate the chain icon in the chart.
2. Click the Analysis chip in the pipeline.
3. Change the analysis in either toolbar.
4. Observe all linked views update.
5. Duplicate the workspace and choose a new source.
```

Because the targets are semantic presentations, tutorial steps need not depend on brittle DOM selectors alone.

---

23. Worked census example

This walkthrough uses the repository's actual fixture and welcome-document conventions.

23.1 Build the analysis

Source:

```
kind: dataset
drop: lab
dataset: census
version: 2
path: rows.csv
```

Relevant fields:

```
region       nominal
population   quantitative
area_km2     quantitative
station_id   nominal
```

Transform:

```
aggregate-population
    groupBy: region
    measure:
        name: population_total
        function: sum
        field: population
```

Root view:

```
mark: bar
x: region
y: population_total
color: region
```

This is the repository-equivalent of "population by city."

23.2 Create the workspace

Open:

- Dataset;
- Pipeline;
- Table;
- Encoding;
- Chart.

Each receives a private `primary` analysis binding selecting the same document initially.

This initial state is not yet linked:

```
binding D → doc census-bars
binding P → doc census-bars
binding T → doc census-bars
binding E → doc census-bars
binding C → doc census-bars
```

The selected values happen to be equal, but changing one selector would not change the others.

23.3 Link all compatible views

From the chart's chain menu, choose:

```
Add all compatible views here
```

The command collects the five `primary` analysis ports and attaches them to one new binding:

```
binding A "Regional population" → doc census-bars
```

The bar shows:

```
[⛓ Regional population · 5]
```

23.4 Switch to another existing analysis

The user opens the pipeline selector and chooses `Population and land area`, another existing document.

The binding becomes:

```
binding A → doc census-scatter
```

All five views switch:

- Dataset remains census because that document uses the same source.

- Pipeline becomes empty or shows that document's transforms.
- Table shows row-level census output.
- Encoding shows point mark with `area_km2` and `population`.
- Chart becomes a scatter plot.

No peer events are sent.

23.5 Switch by clicking another pipeline

A pipeline tile elsewhere presents:

```
{
  type: "pipeline",
  value: {
    docId: "climate-temperature",
    terminal: ...
  }
}
```

Its action menu offers:

```
Use owning analysis in “Regional population”
```

PBUI converts the pipeline to its owning `doc`. The binding changes to the climate document. All five views now show the climate analysis.

23.6 Duplicate the workspace independently

The user chooses **Duplicate workspace**.

The full fork produces:

```
original:
    workspace W1
    views V1..V5
    binding A
    doc D1

copy:
    workspace W2
    views V6..V10
    binding B
    doc D2
```

`D2` is a cloned census analysis. Editing it or rebinding B does not affect W1.

23.7 Retarget the copy to a same-schema version

The user chooses census version 3 with the same field names and types.

Preflight classifies it as exact. The preserving source replacement succeeds without a mapping dialog.

The copied document keeps:

- aggregate transform;
- `population_total` alias;
- bar mark;
- x/y/color encodings.

Binding B continues to identify the copied linked group and now selects the adapted document.

23.8 Retarget to different field names

Target fields:

```
city          nominal
residents     quantitative
land_area     quantitative
```

The contract requires:

```
region       nominal       dimension
population   quantitative  measure
```

The mapping dialog suggests:

```
region → city
population → residents
```

The adapted transform becomes:

```
groupBy: city
sum residents as population_total
```

The encoding becomes:

```
x: city
y: population_total
color: city
```

The derived alias remains stable.

Compilation succeeds, so the transaction:

1. inserts the new document;
2. rebinds B;
3. records the mapping;
4. emits one trace entry.

The original workspace remains on census version 2.

23.9 Save as a template

The user saves W2 as:

```
Template: Population aggregation workspace
Slot: Main analysis
Blueprint: adapted population bar document
Contract:
    category dimension, nominal, required
    population measure, quantitative, required
```

A future instance asks for one source or existing analysis and reconstructs the five linked ports automatically.

---

24. Implementation roadmap

The work can be staged so that each phase provides product value.

Phase 1 — Complete the current document-binding experience

Goals:

- use product label `Analysis`;
- add group label and count;
- add `Add all compatible views`;
- use a dedicated analysis-port presentation target;
- owner-qualify pipeline steps;
- add PBUI conversions from analysis-owned objects to `doc`;
- add conflict preview for differently selected groups.

This phase can still use the existing denormalized `documentBindingId` reducer internally.

Phase 2 — Fix workspace copy semantics

Goals:

- rename current `cloneSpace` user action to `Mirror workspace`;
- implement full **Duplicate workspace** by reusing bundle graph-copy logic;
- add tests for alias preservation and independence;
- warn when a group crosses the workspace boundary;
- provide one-step undo.

This directly enables the user's "duplicate the same workspace but use different datasets" workflow, even before preserving source retarget exists.

Phase 3 — Add linked dataset view

Goals:

- add `DatasetApp`;
- make it analysis-bound;
- show root source and input requirements;
- add owner-qualified `analysisSource` presentations;
- add source-browser commands that explicitly choose a target binding.

At the end of this phase, all five desired view types can join one group.

Phase 4 — Normalize subject bindings and app ports

Goals:

- add `bindings` store;
- replace `docBound` with port descriptors;
- migrate `AppView.documents`;
- change selectors and `AnalysisBar` to resolve through bindings;
- add garbage collection and deletion invariants;
- update portable format;
- maintain old-import compatibility.

Do this before adding a second subject kind.

Phase 5 — Exact-schema analysis fork

Goals:

- create pure preserving source-replacement helper;
- add schema-only candidate query;
- clone a document;
- preserve transforms and encodings only when every required field matches exactly;
- compile before commit;
- atomically insert and rebind;
- add revision guard and undo.

This delivers a safe first version without a mapping UI.

Phase 6 — Mapping contracts

Goals:

- derive source-origin requirements and use sites;
- classify compatibility;
- implement mapping UI;
- rewrite typed field references;
- preserve derived aliases;
- cache compatibility;
- record mapping metadata;
- handle schema drift.

Phase 7 — Parameterized templates

Goals:

- introduce binding table and template slots in portable format;
- expose one slot per linked group;
- save blueprints and contracts;
- instantiate from existing analysis or source;
- support optional slots;
- migrate existing templates as fixed snapshots.

Phase 8 — Remote protocol and collaboration

Goals:

- add subject-binding records and port references to protobuf;
- add atomic domain mutations;
- implement expected-revision conflict handling;
- preserve links in remote round trips;
- support collaborative group management and source fork.

Phase 9 — Generalized linked subjects

Only after analysis linking is stable, consider:

- filter sets;
- time ranges;
- row selections;
- cursors;
- color scales.

Each should be a separate subject kind with explicit app ports and conversion semantics.

---

25. Codebase change map

The following map identifies likely modification points. It is not a patch specification; exact filenames may be adjusted to preserve the repository's layering rules.

25.1 Generic PBUI

`src/presentation/types.ts`

Potential additions:

- optional command-start/partial-command verb conventions;
- no change required for the basic selector and conversion model.

`src/presentation/createPbui.tsx`

Potential additions:

- public API for beginning a serializable partial command;
- input-context metadata for source presentation and missing argument.

`src/presentation/registry.ts`

Existing action rules and conversions are sufficient for most link behavior.

Potential additions:

- diagnostics for ambiguous equal-cost conversions when desired;
- optional conversion explanation for command previews.

New or product-level `PbuiCommandHost`

Prefer this in Datalab unless the missing-argument protocol becomes generic enough for PBUI core.

Responsibilities:

- receive a begin-command verb;
- establish accept context;
- resolve conversion;
- dispatch completed verb;
- manage cancellation/focus.

25.2 Presentation product layer

`packages/datalab-ui/src/pbui/types.ts`

Change:

- `step: string` → `step: StepRef`;
- add analysis binding/port, pipeline, encoding, owner-source, and relation refs;
- narrow ambient fallbacks.

`packages/datalab-ui/src/pbui/registry.ts`

Add:

- descriptors;
- identities;
- conversions to `doc`;
- action rules for link, rebind, inspect, fork.

`packages/datalab-ui/src/pbui/descriptors/step.ts`

Use `ref.docId`, not `env.activeDocId`.

`packages/datalab-ui/src/pbui/descriptors/source.ts`

Separate catalog-source actions from owner-qualified analysis-source actions. Remove or de-emphasize implicit active-document targeting.

`packages/datalab-ui/src/pbui/verbs.ts` and verb application

Add serializable link/rebind/fork command verbs.

25.3 Layout and binding state

`packages/datalab-ui/src/store/layout.ts`

Near-term:

- group names;
- explicit merge winner;
- add-all command;
- mirror naming.

Normalized phase:

- `SubjectBinding` records;
- view port references;
- reducers for attach/detach/merge/rebind;
- binding GC;
- membership selectors.

A separate `store/subjects.ts` may reduce file size, provided imports do not create a cycle.

`packages/datalab-ui/src/appkit/registry.ts`

Replace `docBound` with declared ports. Provide compatibility helpers during migration.

`packages/datalab-ui/src/components/molecules/DocBar`

Rename/evolve to `AnalysisBar`.

Present:

- analysis document;
- analysis port;
- binding chip.

Keep tile-title presentation separate.

25.4 Applications

Chart, table, pipeline, encoding apps

Resolve the selected document through:

```
view.bindings.primary → binding → documentId
```

Pass port metadata to `AnalysisBar`.

Pipeline panel

Present owner-qualified step and pipeline references.

Encoding app/panel

Present the owner-qualified encoding object.

Table app/panel

Present the output relation with owner document.

New `DatasetApp`

Read and present root source, schema contract, and retarget commands.

`SourceApp`

Remain a catalog. Add explicit commands targeting an analysis binding.

25.5 Analysis model

`model/graphicAuthoring.ts`

Keep current destructive `replaceDocumentSource`.

Add a preserving pure helper with a distinct name, for example:

```
adaptDocumentToSource(...)
```

Do not overload one function with `reset: boolean`.

New `model/analysisContract.ts`

Responsibilities:

- derive field requirements;
- classify source compatibility;
- rank mappings;
- rewrite owner-context field references;
- serialize portable contracts.

Compiler integration

Expose enough resolved provenance to distinguish source-origin and derived references.

25.6 Analysis resources

`appkit/AnalysisProvider.tsx`

Centralize per-document compiled state and default initialization.

`appkit/analysisCoordinator.ts`

Retain execution coalescing. Add compatibility/resource integration only where it preserves its focused responsibility.

API client

Add schema-only source query or formalize an existing endpoint suitable for candidate checks.

25.7 Workspace copying and templates

`store/bundles.ts`

Extract or reuse graph collection/hydration for in-memory workspace fork.

`model/portable.ts`

Add explicit portable bindings and slots in a version bump.

`store/templates.ts`

Store template mode and slots. Preserve old bundles as fixed snapshots.

Workspace menu verbs

Add:

- mirror;
- duplicate/fork;
- duplicate for another source;
- save parameterized template.

25.8 Persistence and remote

`store/persist.ts`

Add migration and normalized validation.

`remote/codec.ts`

Encode/decode bindings and ports. Remove the explicit link-loss limitation after protocol generation.

Protobuf and generated code

Extend `AppView` and workbench state with typed subject bindings. Add mutation messages and regenerate TypeScript and Go outputs.

---

26. Testing strategy

The feature crosses graph identity, UI selection, compilation, and persistence. Tests should emphasize invariants and alias relationships.

26.1 Binding reducer tests

Test:

- private binding creation;
- merge with same document;
- merge with different documents and explicit winner;
- attach port;
- detach preserves current value;
- rebind changes all consumers by reference;
- unreferenced binding GC;
- invalid subject-kind rejection;
- stale revision rejection;
- document deletion repair.

26.2 Property-based binding graph tests

Generate random:

- views;
- app port declarations;
- bindings;
- document selections;
- link/detach/rebind operations.

Assert after every operation:

- references resolve;
- kinds match;
- one value per binding;
- detach preserves value;
- no unreferenced records after GC;
- equivalent ports remain equivalent.

26.3 PBUI tests

Test:

- analysis-port identity;
- prepared selector called once per accept operation;
- repeated occurrences use semantic memoization;
- exact clicked occurrence is returned;
- ineligible current-group ports are rejected;
- target removed before commit is rejected;
- pipeline/encoding/relation/step conversions resolve to owner doc;
- plain catalog source does not convert to doc;
- action-rule shadowing by stable ID.

26.4 Owner qualification regression tests

Render two pipeline panels for documents α and β with identical step IDs.

Open the step menu in β and verify every verb carries `docId: β`.

This test should fail under the current bare-string design.

26.5 App integration tests

Mount linked chart, table, pipeline, encoding, and dataset views.

Rebind once and assert each reads the same new document.

Test that view-local state such as table sorting and chart size does not become shared merely because the analysis is shared.

26.6 Source adaptation unit tests

Exact schema

Preserve transforms and encodings.

Renamed fields

Apply explicit mapping and preserve derived output alias.

Missing required field

Return `mapping-required` or `incompatible`, with use-site diagnostics.

Ambiguous candidate

Do not guess.

Derived field

Do not rewrite `population_total` when source `population` changes.

Nested field references

Exercise every transform and view location.

Compile failure

Return failure with no mutated input document.

Revision race

Reject stale plan at commit.

Stream policy

Enforce supported source-kind rules.

26.7 Workspace graph-copy tests

Construct a workspace with:

- two placements of one logical view;
- five views sharing one binding;
- two bindings selecting one document;
- a group with one member outside the workspace.

Fork and assert:

- all runtime IDs are new;
- duplicate placements still share one cloned view;
- linked members share one cloned binding;
- shared source document aliases become one cloned document;
- no cloned identity points back to original;
- external group member remains on original;
- copied members form an independent group.

Property-based graph cloning is particularly valuable here.

26.8 Portable round-trip tests

Test:

- current format;
- migration from v3 binding indices;
- normalized ports;
- template slots;
- optional slots;
- fixed documents;

- malformed indices;
- dangling references;
- unsupported subject kinds;
- credential audit.

26.9 Remote codec tests

Test:

- bindings survive encode/decode;
- old remote workbench decodes to private bindings;
- expected revisions;
- atomic fork mutation;
- dangling document rejection;
- unknown subject kind handling;
- Go and TypeScript generated schema compatibility.

26.10 Performance tests

Measure:

- one prepared port-set build per accept;
- no row evaluation during link hover/select;
- one compile per document revision across linked views;
- in-flight execution coalescing;
- compatibility cache hit;
- no million-row fetch for schema checks;
- bounded member-manager rendering.

26.11 Accessibility tests

Test:

- chain state accessible names;
- keyboard accept/cancel;
- focus restoration;
- status announcements;
- conflict dialog labels;
- group member list semantics;
- no color-only distinction.

26.12 Storybook scenarios

Add stories:

1. Private analysis port.
2. Five linked views.
3. Link target highlighting.
4. Conflict merge preview.
5. Unlink without visual change.
6. Dataset app exact compatibility.
7. Mapping-required state.
8. Invalid schema drift.
9. Mirror versus fork workspace.
10. Template slot instantiation.

---

27. Rejected alternatives

27.1 Peer-to-peer view links

Rejected because they create event graphs, ordering problems, and partial updates.

Use one shared subject.

27.2 Global active analysis

Rejected because it prevents independent groups and makes visible-object actions depend on focus.

Use owner-qualified presentations and explicit bindings.

27.3 Copy pipeline and encoding into each tile

Rejected because each tile would own a divergent copy of one composition.

Keep the analysis document as the source of truth.

27.4 Treat pipeline, table, encoding, and chart as one subtype hierarchy

Rejected because they are not substitutable instances of one object type. They are different objects or projections owned by one analysis.

Use conversions to the owning analysis, not false subtyping.

27.5 Convert any catalog source to an analysis

Rejected because a source has no unique owning analysis.

Use a two-argument fork/retarget command.

27.6 Store member lists inside bindings and binding IDs inside views

Rejected because it creates two mutable sources of membership truth.

Persist port references; derive reverse membership.

27.7 One shared-group ID for all future interaction state

Rejected because analysis selection, filters, cursor, row selection, and zoom need independent coupling.

Use typed subjects and named ports.

27.8 Silent best-effort field mapping

Rejected because an incorrect chart can look plausible.

Require explicit choice for ambiguous mappings and compile before commit.

27.9 Destructive source reset under the label “switch dataset”

Rejected because it discards work unexpectedly.

Keep reset as a clearly named operation and add fork/adapt.

27.10 Call geometry-only cloning “Duplicate workspace”

Rejected because the copy changes with the original.

Call it Mirror. Make Duplicate a graph fork or present explicit choices.

27.11 Split pipeline and encoding into first-class reusable artifacts immediately

Rejected as the first step because the current `GraphicDocument` already gives coherent ownership and the main requested workflow does not require independent artifact versioning.

Reconsider only when users need to share one pipeline across many documents while independently changing it, compose pipelines as libraries, or version encodings separately.

---

28. Open design choices

Several decisions require product judgment. None blocks the core architecture.

28.1 Merge winner default

Options:

- initiating/source group wins;
- target wins;
- larger group wins;
- ask only when values differ.

Recommendation: source wins for same-value/no-conflict cases; ask when values differ.

28.2 User-facing term

Candidates:

- Analysis;
- Composition;
- Document;
- Graphic;
- Work item.

Recommendation: **Analysis**. It communicates that source, pipeline, table result, and visualization belong together. Keep internal names until a broader refactor is justified.

28.3 Default duplicate semantics

Recommendation:

- primary menu item: **Duplicate workspace** → full fork;
- secondary: **Mirror workspace**;
- advanced: **Duplicate layout with shared analyses**.

This aligns the default with user expectations and preserves the useful current behavior under a precise name.

28.4 Forked document naming

Examples:

```
Population by region — copy
Population by region · census v3
```

```
Population by region · city demographics
```

Recommendation: derive a provisional name from source and let the user edit it in the preflight.

28.5 Orphan document policy

After undo or workspace deletion, a cloned document may have no bindings.

Options:

- immediate garbage collection;
- retain in recent analyses;
- prompt;
- TTL cleanup.

Recommendation: retain as a recent analysis for the session, with explicit cleanup, unless storage scale demands eager GC. Bindings themselves can be eagerly garbage-collected.

28.6 In-place retarget permissions

In-place retarget changes every binding that selects the same document.

Recommendation: make it advanced and show all affected groups. Default to fork.

28.7 Cross-workspace linking

The data model can support it. The interaction should default to visible/current workspace targets because cross-workspace links are harder to understand.

A group manager can expose external members explicitly.

28.8 Empty bindings

Allowing `documentId: null` is useful for optional template slots and deleted-document repair.

Required app ports should render a clear empty state rather than fail.

28.9 Multiple sources in one document

The current authoring helpers often assume a root/first source. Future joins require several source-bound ports or one analysis binding plus source-node-specific presentations.

The normalized subject architecture does not require each source to be a top-level UI binding. The analysis remains the selected composition; a join editor can expose source-node presentations within it.

28.10 Whether a pipeline becomes a standalone object later

A standalone pipeline artifact becomes useful when users need:

- library reuse across analyses;
- pipeline versioning independent of charts;
- parameterized pipelines;
- several root views over one transform graph;
- explicit pipeline ownership and permissions.

Until then, “pipeline presentation converts to owner document” is simpler and coherent.

---

29. Acceptance criteria

A first complete release should satisfy the following observable criteria.

29.1 Linking

- A chart, table, pipeline, encoding, and dataset view can be placed in one linked group.
- The group has a visible chain indicator and member count.
- Changing the analysis from any member updates every member.
- Unlinking one member leaves it showing the same analysis.
- Linking groups with different analyses shows an explicit winner preview.
- Link selection uses PBUI presentation input, keyboard operation, and semantic identity.

29.2 Object correctness

- Pipeline steps and other owned objects target their actual document.
- Clicking a pipeline, encoding, relation, source-in-analysis, or step can select its owning analysis through PBUI conversion.
- A catalog source is never mistaken for an existing analysis.

29.3 Source reuse

- A user can fork a linked analysis onto a same-schema source without losing pipeline or encoding.
- A renamed-field source opens a mapping flow.
- Ambiguous mappings are not silently applied.
- Compilation failure commits nothing.
- The original analysis remains intact.
- Undo returns the group to the original analysis.

29.4 Workspace reuse

- Mirror workspace preserves shared logical views.
- Duplicate workspace produces independent views, bindings, and documents.
- Editing or rebinding the duplicate does not affect the original.
- Internal linked-group topology survives duplication.
- Multiple placements of one view remain multiple placements of one cloned view.

29.5 Templates

- A linked group can be exposed as one named template slot.
- Instantiation asks once for the slot.

- The slot may bind an existing analysis or adapt a blueprint to a source.
- Portable import/export preserves slots and group topology.
- Old snapshot templates remain readable.

29.6 Persistence and collaboration

- Local reload preserves links.
- Portable round trip preserves links.
- Remote workbench round trip preserves links.
- Stale concurrent updates are detected.
- Document deletion cannot leave silent dangling bindings.

29.7 Performance

- Link target hit testing performs no row evaluation.
- Same-document linked views share in-flight analysis execution.
- Source compatibility uses schema-only access.
- A group rebind updates one normalized subject record.
- Large workspaces remain responsive.

---

30. Glossary

Analysis
The product-facing name recommended for a `GraphicDocument`: source, transforms, relation, visual specification, and parameters as one coherent composition.

Analysis binding
A stable subject record whose value is the selected analysis document.

Analysis port
A named connection point on an application view that refers to an analysis binding.

Application object
A semantic domain object represented in the UI.

Binding group
The derived set of ports that reference one binding.

Conversion
A typed interpretation from one presentation object to another, such as pipeline to owning analysis.

Detach
Give a port a private binding initialized from its current shared binding.

Fork
Clone identities and content while preserving internal graph relationships and independence from the original.

Input context
A temporary state in which PBUI accepts presentations satisfying a selector or convertible target type.

Logical view
One open application instance, independent of workspace placement.

Mirror
A second layout tree that presents the same logical views and therefore shares their bindings and documents.

Owner-qualified reference
A presentation value that carries both local object identity and its owning analysis, such as `{ docId, stepId }`.

Placement
A workspace rectangle that presents a logical view.

Presentation
A visible occurrence associated with an application object and presentation type.

Retarget
Change an analysis source while attempting to preserve and adapt downstream setup.

Schema contract
The source-field and capability requirements implied by an analysis or declared by a template.

Subject
A typed shared selection or interaction value observed by application ports.

Template slot
A named subject input in a parameterized workspace template.

---

31. References

Repository paths analyzed

- `src/presentation/types.ts`
- `src/presentation/selectors.ts`
- `src/presentation/registry.ts`
- `src/presentation/conversions.ts`
- `src/presentation/createPbui.tsx`
- `packages/datalab-ui/src/store/layout.ts`
- `packages/datalab-ui/src/store/layoutTree.ts`
- `packages/datalab-ui/src/store/world.ts`
- `packages/datalab-ui/src/store/bundles.ts`
- `packages/datalab-ui/src/store/templates.ts`
- `packages/datalab-ui/src/store/persist.ts`
- `packages/datalab-ui/src/model/graphic.ts`
- `packages/datalab-ui/src/model/graphicAuthoring.ts`
- `packages/datalab-ui/src/model/portable.ts`
- `packages/datalab-ui/src/model/transformEditor.ts`
- `packages/datalab-ui/src/appkit/registry.ts`
- `packages/datalab-ui/src/appkit/AnalysisProvider.tsx`
- `packages/datalab-ui/src/appkit/analysisCoordinator.ts`
- `packages/datalab-ui/src/apps/ChartApp/ChartApp.tsx`
- `packages/datalab-ui/src/apps/TableApp/TableApp.tsx`
- `packages/datalab-ui/src/apps/PipelineApp/PipelineApp.tsx`
- `packages/datalab-ui/src/apps/EncodingApp/EncodingApp.tsx`
- `packages/datalab-ui/src/apps/SourceApp/SourceApp.tsx`
- `packages/datalab-ui/src/components/molecules/DocBar/DocBar.tsx`
- `packages/datalab-ui/src/components/organisms/PipelinePanel/PipelinePanel.tsx`
- `packages/datalab-ui/src/pbui/types.ts`
- `packages/datalab-ui/src/pbui/registry.ts`
- `packages/datalab-ui/src/pbui/descriptors/step.ts`
- `packages/datalab-ui/src/pbui/descriptors/source.ts`
- `packages/datalab-ui/src/remote/codec.ts`
- `proto/hyperslop/pbui/workbench/v1/workbench.proto`
- `packages/datalab-ui/src/fixtures/census.json`
- `packages/datalab-ui/src/demo/welcome.ts`

CLIM background

The following official LispWorks CLIM documentation is useful background for the presentation/input-context/translator/command model:

- CLIM presentation concepts
- Presentation types
- Presentation translators
- Defining presentation translators
- Commands
- Command processing
- CLIM glossary

The preceding design borrows the separation of application object, presentation type, input context, conversion/translator, and command. It does not claim API compatibility with CLIM.

---

32. Final recommendation

The enhanced codebase has already crossed the most important conceptual boundary: a chart and a pipeline can remain distinct logical applications while sharing a document-selection subject.

The next step should not be a larger collection of tile-to-tile synchronization handlers. It should be a disciplined generalization of that subject.

The recommended sequence is:

1. Treat `GraphicDocument` as the selected **Analysis**.
2. Present a named analysis port in every dataset, pipeline, table, encoding, and chart tile.
3. Use PBUI selectors, semantic identity, conversions, and action rules to link ports and rebind groups.
4. Fix ownerless analysis-owned presentations, beginning with pipeline steps.
5. Rename the current geometry-only copy to **Mirror workspace** and make ordinary duplication a full graph fork using the existing portable bundle machinery.
6. Add a document-bound dataset application while retaining the singleton source catalog.
7. Add a preflighted **Fork analysis onto source** operation that preserves transforms and encodings, uses schema contracts, requests mappings, compiles before commit, and is undoable.
8. Normalize bindings and app ports before adding other linked state kinds.
9. Extend templates with one slot per linked analysis group.
10. Extend the remote protocol so that bindings are first-class collaborative state.

This architecture gives users the desired experience:

```
Build one workspace for a type of work.
Link its analytical views once.
Switch the whole workspace by choosing an analysis anywhere.
Duplicate it without hidden aliases.
Apply the setup to another source without discarding the pipeline.
Save the linked topology as a reusable template with one meaningful input.
```

It also preserves the main strengths of the current PBUI and Datalab design: semantic presentations, serializable actions, explicit identity, pure graph transformations, deterministic state transitions, and a single coherent analytical composition.